DELTA – Střední škola informatiky a ekonomie, s.r.o.
Ke Kamenci 151, Pardubice

SubmiTheses

Systém pro správu a odevzdávání maturitních prací

Bohdan Bodnarchuk
4.A Informační technologie

2025/2026

Zadání maturitního projektu z informatických předmětů

Jméno a příjmení: *Bohdan Bodnarchuk*

Pro školní rok: *2025/2026*

Třída: *4. A*

Obor: *Informační technologie 18-20-M/01*

Téma práce: SubmiTheses

Online systém pro správu a archivaci maturitních projektů

Vedoucí práce: *RNDr. Jan Koupil, Ph.D.*

Způsob zpracování, cíle práce, pokyny k obsahu a rozsahu práce:

Cíl projektu:

Cílem projektu je navrhnout a vytvořit online aplikaci pro správu životního cyklu maturitních projektů. Systém bude sloužit jako nástroj pro evidenci, ukládání, kontrolu a archivaci studentských projektů a posudků. Důraz bude kladen na přehlednost, flexibilitu, správu přístupových práv a dlouhodobou udržitelnost systému ve školním prostředí.

Specifikace projektu:

1. Analýza a návrh:
    - Bude provedena analýza současné praxe (např. ukládání projektů v domovských složkách) a existujících řešení v oblasti správy závěrečných prací.
    - Budou definovány role uživatelů (student, vedoucí, oponent, administrátor) a jejich oprávnění.
    - Bude navržena datová struktura systému a architektura aplikace.
2. Evidence a správa projektů:

    - Bude implementováno přehledné rozhraní pro evidenci všech maturitních prací, včetně přiřazení řešitele, vedoucího a oponenta.
    - Systém umožní změnu těchto přiřazení i v průběhu školního roku.
    - Každý student bude mít možnost nahrát text práce (povinně ve dvou formátech), anotaci, a volitelné přílohy (obrazové přílohy, odkazy apod.).
3. Odevzdávání a uzamykání:
    - Bude implementována možnost uzamykání a odemykání prací pro editaci – ručně i automaticky podle nastaveného data.
    - Po uzamčení bude studentům znemožněna další úprava.
4. Zadávání posudků:
    - Vedoucí a oponent budou moci po odevzdání projektu vkládat své posudky.
    - Po uplynutí lhůty pro vložení posudků již nebude možné tyto dokumenty měnit, ale studenti je budou moci vidět.
5. Správa přístupů a identit:
    - Účty budou navázány na školní doménu (např. Office 365), přičemž systém se musí umět vypořádat i s uživateli mimo doménu.
    - Systém bude reflektovat možnost, že jedna osoba je vedoucím i oponentem různých projektů, a bude ji informovat o nevyřízených úkonech (např. chybějící posudky).
6. Administrátorské rozhraní:
    - Bude vytvořeno rozhraní pro správu uživatelů, projektů a aktuálního ročníku.
    - Administrátor bude mít přístup ke statistikám a přehledům (např. kolik studentů ještě nemá zadání, kolik chybí posudků apod.).
7. Zveřejnění vybraných prací:
    - Bude umožněno označit vybrané projekty jako veřejné a zpřístupnit je i nepřihlášeným uživatelům.
    - Takto označené práce budou mít samostatnou veřejnou stránku s anotací, soubory, odkazy a posudky, a budou sloužit jako výkladní skříň školy.
8. Testování a dokumentace:
    - Bude provedeno testování funkcionalit a uživatelského rozhraní v typických scénářích.
    - Bude vytvořena uživatelská dokumentace (návody, popis funkcí) i technická dokumentace systému.
9. Prezentace a obhajoba:
    - Bude připravena prezentace projektu s ukázkou funkcí systému.

    - V rámci obhajoby bude popsán návrh architektury, vývojový proces a dosažené výstupy.

Stručný časový harmonogram:

Září: Analýza existujících řešení, definice požadavků, výběr technologie

Říjen: Návrh datového modelu, návrh architektury, návrh rolí a oprávnění

Listopad: Implementace základní funkcionality: evidence projektů, správa uživatelů

Prosinec: Implementace odevzdávání projektů, příloh, anotací, uzamykání a řízení přístupů

Leden: Posudky, role vedoucího/oponenta, uživatelské upozornění na neaktivitu, Základní administrátorské přehledy

Únor: Implementace veřejných stránek projektů, zpřístupnění výběru prací, testování a ladění

Březen: Tvorba dokumentace a prezentace, závěrečné testy

Požadované výstupy:

- Funkční webová aplikace pro správu maturitních projektů
- Uživatelské rozhraní pro všechny role (student, vedoucí, oponent, administrátor)
- Možnost zveřejnění vybraných projektů
- Uživatelská a technická dokumentace
- Prezentace a demonstrační ukázka

Hodnocení:

- Projekt bude hodnocen na základě:
- Úplnosti a funkčnosti systému
- Srozumitelnosti a použitelnosti rozhraní pro cílové skupiny
- Schopnosti řídit přístupy a správu rolí
- Kvality dokumentace a prezentace
- Přínosu pro školní praxi

Prohlašuji, že jsem maturitní projekt vypracoval(a) samostatně, výhradně s použitím uvedené literatury.

V Pardubicích dne ……

…….

Resumé

Práce se zabývá vývojem webového informačního systému pro správu a odevzdávání maturitních projektů. Systém umožňuje studentům nahrávat práce, učitelům je hodnotit pomocí konfigurovatelných škál a administrátorům spravovat uživatele, předměty a akademické roky. Backend je postaven na Node.js s Express a PostgreSQL, frontend využívá Next.js s React.

Klíčová slova: webová aplikace, informační systém, maturitní projekt, Next.js, Express.js, PostgreSQL, hodnocení, správa dokumentů

Abstract

This thesis focuses on the development of a web-based information system for managing and submitting graduation projects. The system allows students to upload their work, teachers to evaluate submissions using configurable grading scales, and administrators to manage users, subjects, and academic years. The backend is built on Node.js with Express and PostgreSQL, while the frontend utilizes Next.js with React.

Keywords: web application, information system, graduation project, Next.js, Express.js, PostgreSQL, grading, document management

Obsah

1 Úvod . . . 6
2 Teoretická část . . . 6
2.1 Existující řešení . . . 6
2.2 Použité technologie . . . 7
3 Metodika a vlastní řešení . . . 8
3.1 Architektura systému . . . 8
3.1.1 Diagram případů užití . . . 9
3.2 Datový model . . . 14
3.3 Návrh uživatelského rozhraní . . . 17
3.3.1 Principy návrhu . . . 17
3.3.2 Struktura rozložení . . . 17
3.3.3 Navigace podle rolí . . . 17
3.3.4 Klíčové obrazovky . . . 18
3.3.5 Responzivní chování . . . 22
3.4 Implementace . . . 22
3.4.1 Backendová část . . . 22
3.4.2 Frontendová část . . . 25
4 Dokumentace API . . . 27
4.1 Autentizace . . . 27
4.2 Přehled endpointů . . . 27
5 Uživatelská příručka . . . 30
5.1 Nasazení na server . . . 30
5.1.1 Docker konfigurace . . . 30
5.1.2 Úložiště souborů (Garage) . . . 31
5.1.3 Reverzní proxy (Nginx) . . . 32
5.1.4 Manuální nasazení . . . 33
5.1.5 CI/CD pomocí GitHub Actions . . . 33
5.2 Použití aplikace . . . 33
5.2.1 Přihlášení do systému . . . 34

5.2.2 Přehled projektů . . . 34
5.2.3 Vytvoření projektu . . . 34
5.2.4 Volná témata . . . 35
5.2.5 Detail projektu . . . 35
5.2.6 Správa příloh a odkazů . . . 35
5.2.7 Hodnocení . . . 36
5.2.8 Oznámení . . . 36
5.2.9 Nastavení profilu . . . 36
5.2.10 Administrátorský panel . . . 37
5.2.11 Veřejná galerie . . . 37
6 Bezpečnost . . . 37
6.1 SQL Injection . . . 37
6.2 Cross-Site Scripting (XSS) . . . 38
6.3 Cross-Site Request Forgery (CSRF) . . . 38
6.4 Zabezpečení OAuth . . . 38
6.5 Ochrana hesel . . . 39
6.6 Rate Limiting . . . 39
6.7 Validace vstupních dat . . . 39
6.8 CORS (Cross-Origin Resource Sharing) . . . 39
6.9 Bezpečnost souborů . . . 39
6.10 Autorizace a řízení přístupu . . . 40
6.11 Statická analýza kódu (CodeQL) . . . 40
7 Výsledky . . . 40
8 Diskuse . . . 41
8.1 Zdůvodnění technologických voleb . . . 41
8.2 Co se osvědčilo . . . 42
8.3 Omezení a prostor pro zlepšení . . . 42
8.4 Srovnání s existujícími řešeními . . . 42
9 Závěr . . . 42
Literatura . . . 43

Seznam obrázků 44
Seznam tabulek 44
Přílohy 45
9.1 Příloha A: Zdrojový kód aplikace 45
9.2 Příloha B: Pokyny k nasazení 45

1 Úvod

V dnešní době naše škola nemá jednotné řešení pro správu celého životního cyklu maturitních projektů. Celý proces – od výběru tématu přes průběžné konzultace až po finální odevzdání a hodnocení – probíhá nekoordinovaně prostřednictvím různých nástrojů: e-mailů, sdílených složek a papírových formulářů. Tento přístup je náchylný k chybám, nepřehledný a časově náročný pro všechny zúčastněné strany.

Cílem této práce je vytvořit webový informační systém SubmiTheses, který tento problém řeší komplexním způsobem.

Studenti se mohou přihlásit k tématu ze seznamu volných prací, odevzdávat a spravovat přílohy svého projektu a sledovat stav hodnocení.

Vedoucí prací mohou snadno vytvářet a upravovat témata maturitních projektů, přiřazovat vybrané studenty ze seznamu zájemců, průběžně sledovat vývoj projektu a hodnotit odevzdané práce pomocí konfigurovatelných hodnotících škál.

Administrátoři mají plnou kontrolu nad celým systémem – mohou spravovat uživatelské účty, akademické roky, předměty, hodnotící škály a také přímo upravovat či spravovat jednotlivé projekty.

Systém tak centralizuje veškerou komunikaci a dokumentaci na jednom místě, čímž zvyšuje přehlednost, snižuje administrativní zátěž a minimalizuje riziko ztráty důležitých souborů či informací.

2 Teoretická část

2.1 Existující řešení

Na trhu existuje několik systémů, které částečně pokrývají problematiku správy studentských prací:

- **Moodle** je komplexní systém pro řízení výuky (LMS), který umožňuje odevzdávání úkolů a hodnocení. Jedná se však o obecný nástroj, který není specializovaný na správu maturitních projektů a jejich životního cyklu. Konfigurace pro tento účel by byla složitá a nepřehledná.
- **Google Classroom** nabízí jednoduché rozhraní pro zadávání a odevzdávání úkolů, ale postrádá pokročilé funkce jako konfigurovatelné hodnotící škály, správu témat projektů či přiřazování studentů k vedoucím.

- **GitHub Classroom** je zaměřen především na programátorské úkoly a správu kódu. Není vhodný pro komplexní správu dokumentace a hodnocení maturitních prací.
- **Microsoft Teams** poskytuje komunikační platformu s možností sdílení souborů a týmové spolupráce. Chybí mu však specializované funkce pro správu životního cyklu projektů, konfigurovatelné hodnotící škály a přehledné zobrazení stavu jednotlivých prací.
- **Ruční proces** pomocí e-mailů a sdílených složek je náchylný k chybám, nepřehledný a neumožňuje centrální sledování stavu projektů.

Žádné z existujících řešení plně nepokrývá specifické potřeby naší školy – proto vznikl systém SubmiTheses.

2.2 Použité technologie

Pro vývoj systému byly zvoleny následující technologie:

- **Next.js** – moderní React framework s podporou server-side renderingu, což zlepšuje výkon a SEO. Využívá souborový systém pro routování, což zjednodušuje strukturu projektu.
- **Express.js** – minimalistický a flexibilní Node.js framework pro tvorbu REST API. Umožňuje rychlý vývoj a snadnou integraci middleware.
- **PostgreSQL** – relační databáze vhodná pro strukturovaná data s vazbami (uživatelé, projekty, hodnocení). Zajišťuje konzistenci dat díky ACID vlastnostem.
- **Prisma ORM** – typově bezpečný objektově-relační mapovač, který zjednodušuje práci s databází a poskytuje automatické migrace schématu.
- **Garage** – self-hosted S3-kompatibilní objektové úložiště pro přílohy projektů a profilové obrázky. Běží jako Docker kontejner v rámci infrastruktury aplikace.
- **Redis** – in-memory databáze využívaná pro cachování často používaných dat a jako message broker pro frontu úloh BullMQ.
- **BullMQ** – knihovna pro zpracování úloh na pozadí postavená na Redis. Zajišťuje spolehlivé plánování a provádění časově závislých operací (připomenutí termínů, automatické zamykání projektů).
- **Autentizace a bezpečnost** – systém využívá JWT tokeny pro autentizaci uživatelů a bcrypt pro bezpečné hashování hesel.

Pro stylování uživatelského rozhraní je využit **Tailwind CSS** s komponentovou knihovnou **Flowbite**. Lokalizace aplikace je zajištěna knihovnou **next-intl**, validace vstupních dat probíhá pomocí **Zod** a e-mailové notifikace jsou odesílány přes **nodemailer**.

3 Metodika a vlastní řešení

3.1 Architektura systému

Systém SubmiTheses je navržen jako monorepo obsahující dvě hlavní části – frontend a backend – které spolu komunikují prostřednictvím REST API.

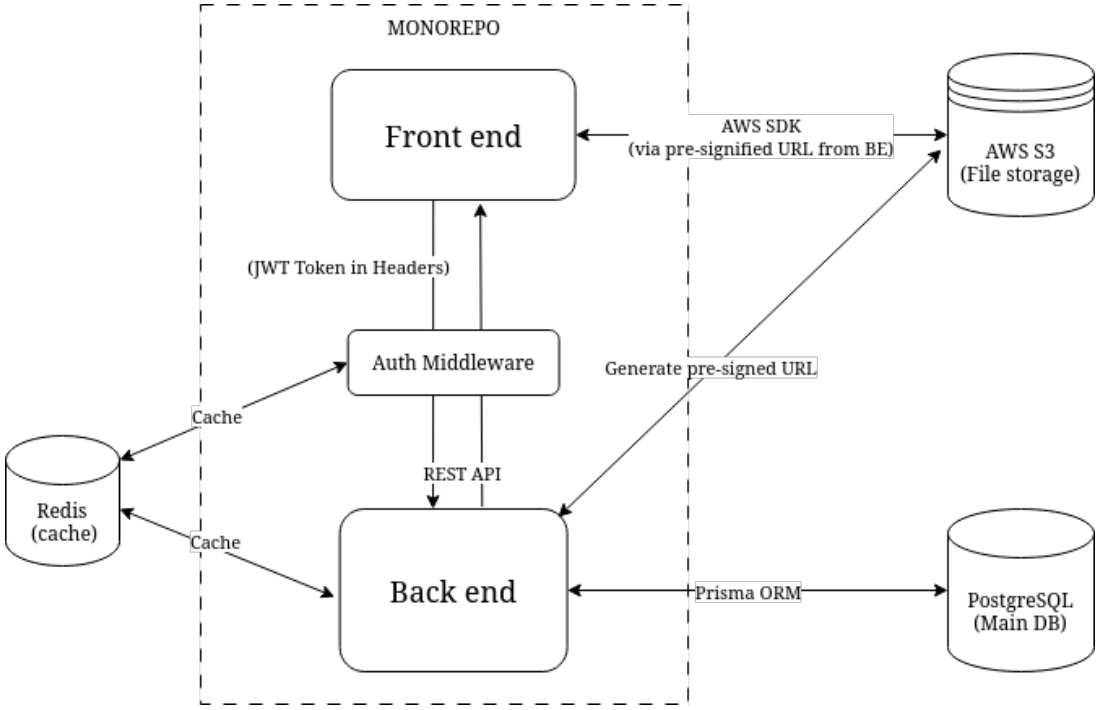
Frontend je postaven na frameworku Next.js a zajišťuje uživatelské rozhraní aplikace. Komunikuje s backendem pomocí HTTP požadavků, přičemž každý požadavek obsahuje JWT token v hlavičce pro autentizaci uživatele.

Backend využívá Express.js a zpracovává veškerou aplikační logiku. Pro přístup k databázi PostgreSQL používá Prisma ORM, který zajišťuje typově bezpečnou práci s daty a automatické migrace schématu. Pro optimalizaci výkonu je implementována cache pomocí Redis, která ukládá často používaná data.

Ukládání souborů je řešeno prostřednictvím self-hosted S3-kompatibilního úložiště Garage. Backend generuje pre-signed URL adresy, které umožňují frontendu nahrávat soubory přímo do S3 bez nutnosti průchodu dat přes server. Tento přístup snižuje zatížení backendu a zrychluje nahrávání velkých souborů.

Fronta úloh pro zpracování na pozadí využívá knihovnu BullMQ s Redis jako brokerem. Systém automaticky plánuje připomenutí blížících se termínů a zamykání projektů po uplynutí deadlinu. Konfigurovatelné dny připomenutí (výchozí 7, 3 a 1 den před termínem) jsou nastavitelné pro každý akademický rok.

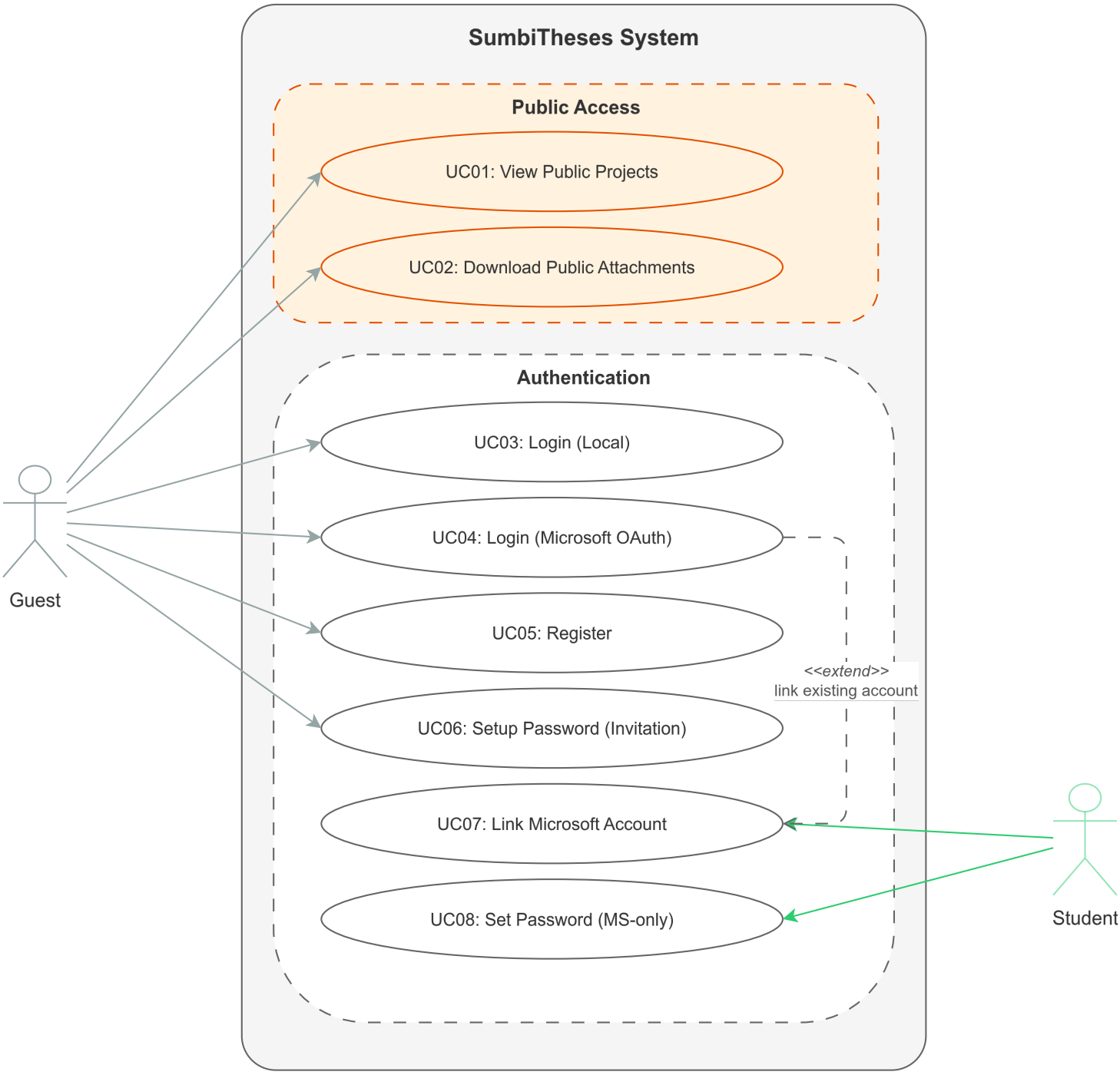
Autentizace je zajištěna pomocí Auth Middleware, který ověřuje JWT tokeny a kontroluje oprávnění uživatelů před zpracováním požadavků. Celková architektura systému je znázorněna na obr. 1.



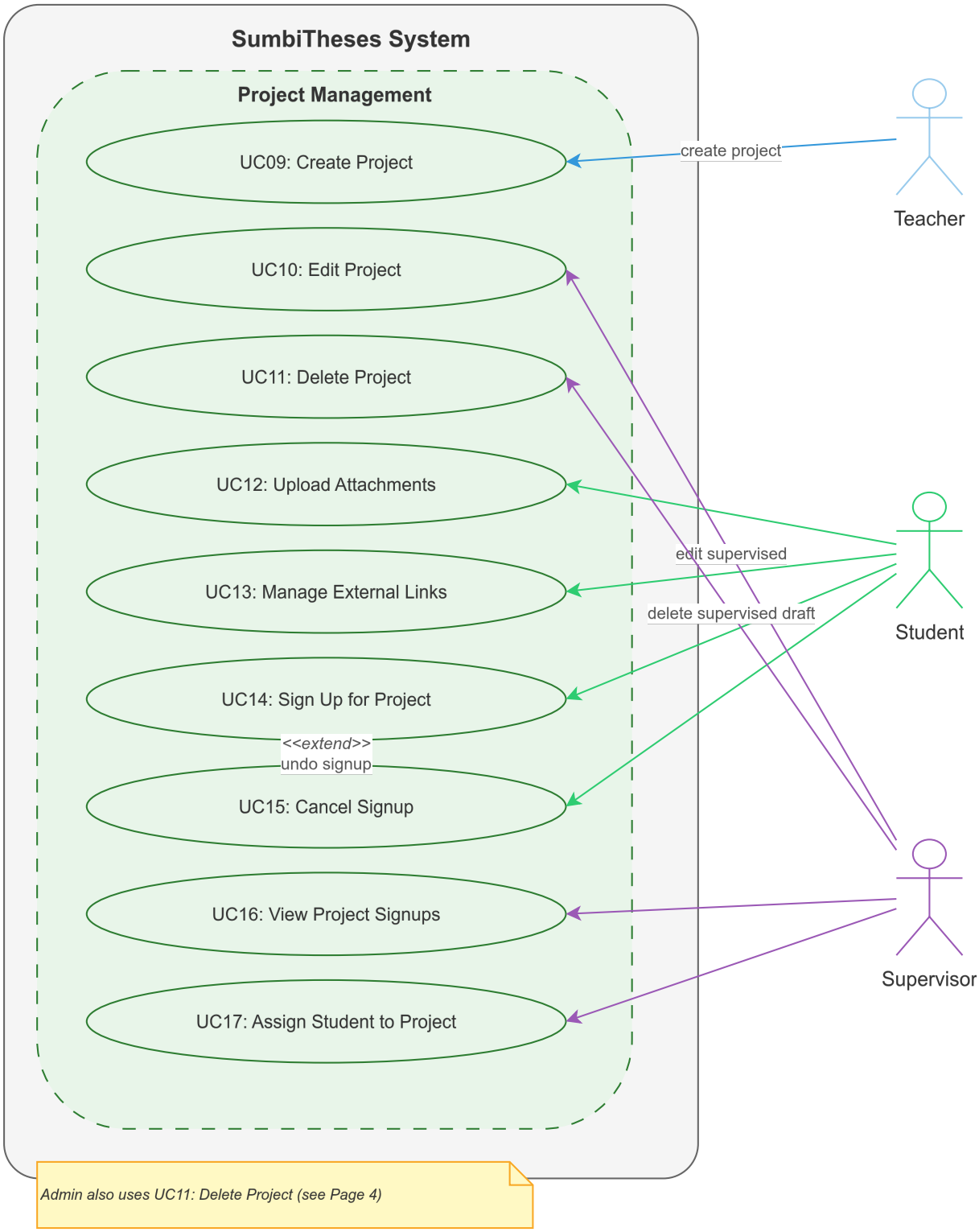
obr. 1: Schéma architektury

3.1.1 Diagram případů užití

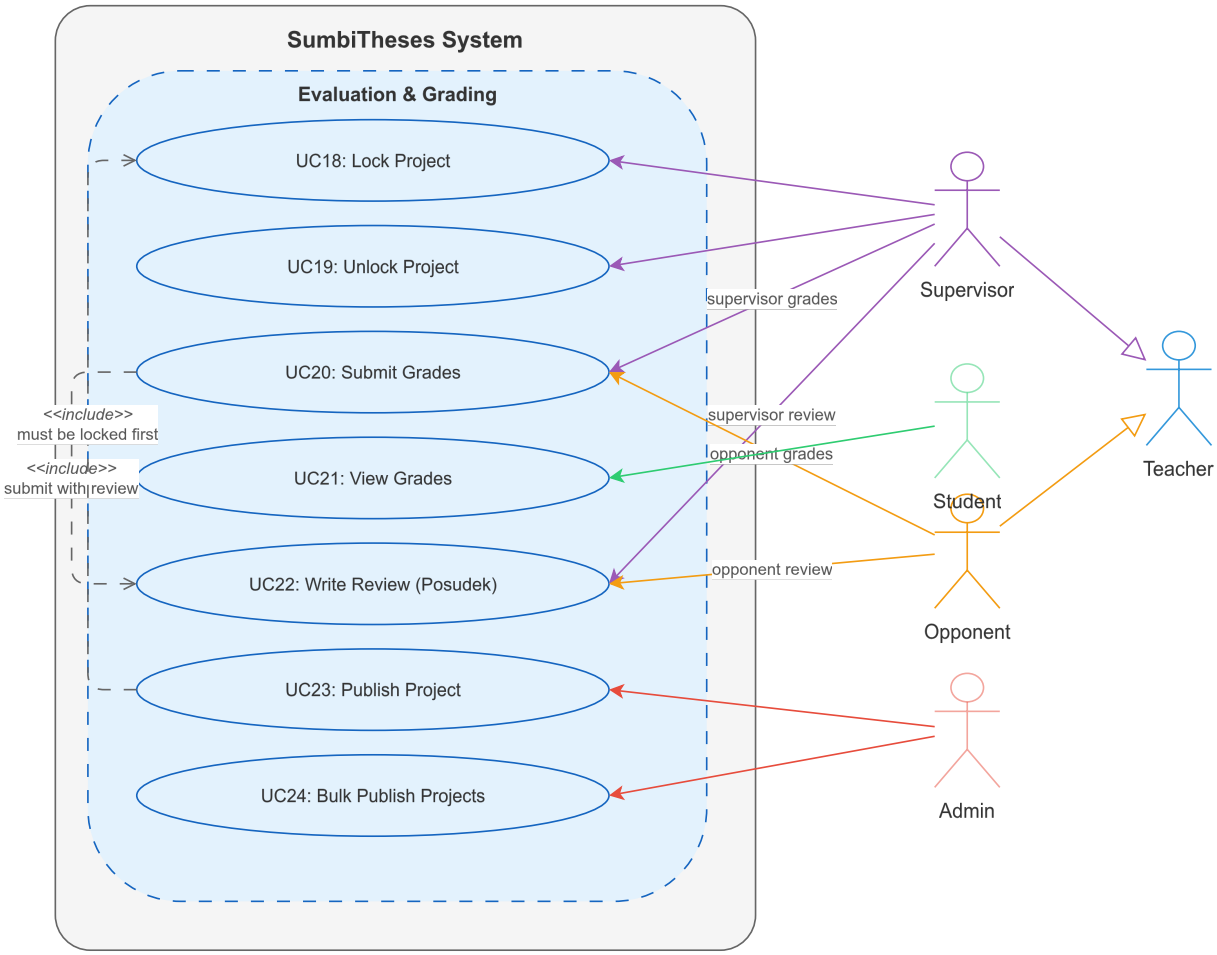
Funkční požadavky systému jsou zachyceny formou diagramu případů užití (Use Case Diagram). Diagram je rozdělen do pěti tematických oblastí podle funkčních celků systému.



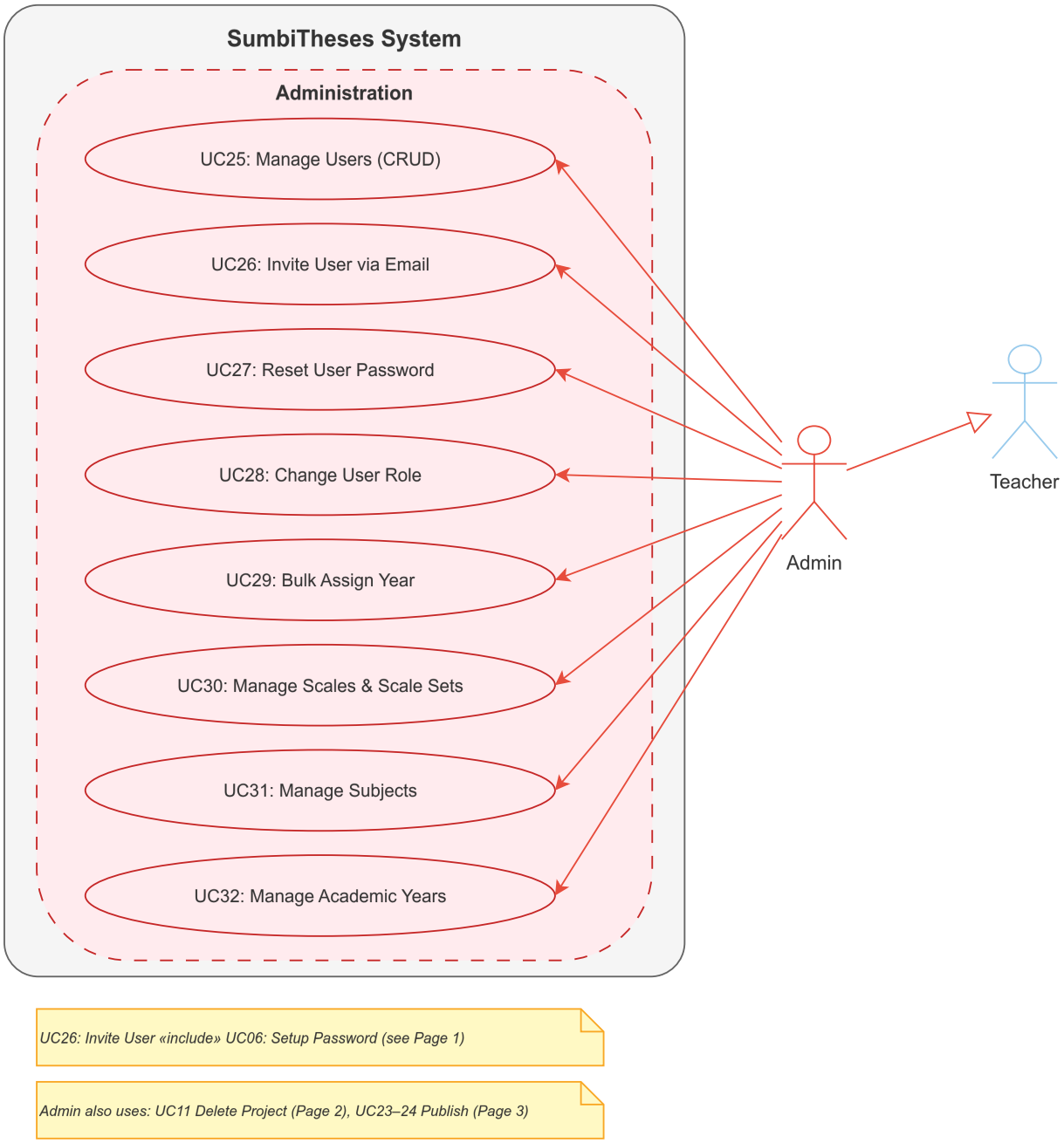
obr. 2: Případy užití — Veřejný přístup a autentizace (UC01–UC08)



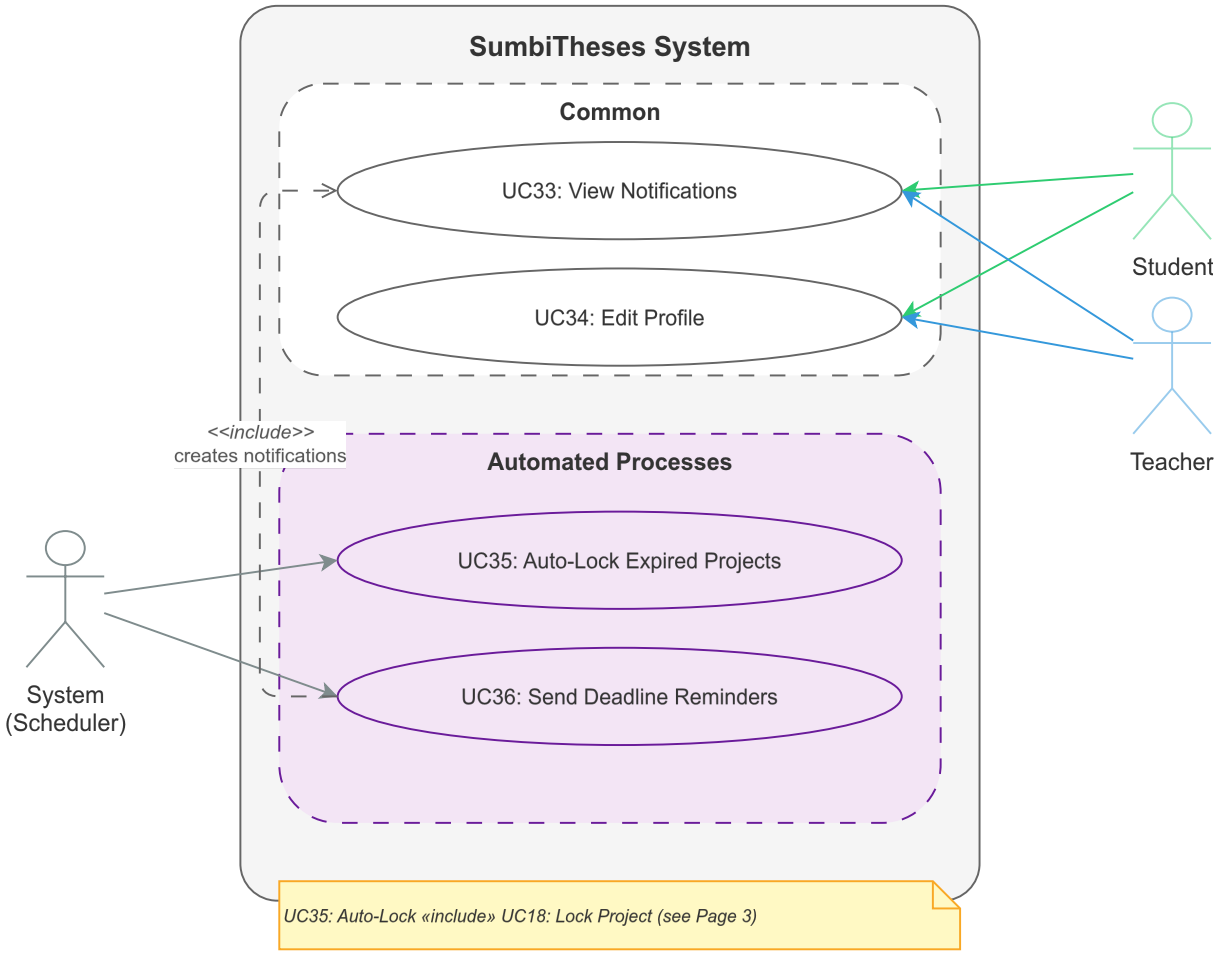
obr. 3: Případy užití — Správa projektů (UC09–UC17)



obr. 4: Případy užití — Hodnocení a klasifikace (UC18–UC24)



obr. 5: Případy užití — Administrace (UC25–UC32)



obr. 6: Případy užití — Společné a automatizované procesy (UC33–UC36)

3.2 Datový model

Databáze systému je postavena na PostgreSQL a obsahuje následující hlavní entity:

Uživatelé (users) – centrální entita pro správu účtů. Každý uživatel má přiřazenou roli (admin, teacher, student), kontaktní údaje a volitelně odkaz na akademický rok. Studenti mají navíc atribut třídy.

Projekty (projects) – hlavní entita reprezentující maturitní práci. Obsahuje název, popis, stav (draft, locked, public) a vazby na studenta, vedoucího a oponenta. Projekt může být uzamčen pro úpravy s uvedením důvodu a času uzamčení.

Popis projektu (project_descriptions) – rozšiřující entita v relaci 1:1 s projektem. Obsahuje detailní specifikaci práce: téma, cíl, harmonogram, požadované výstupy a kritéria hodnocení.

Akademické roky (years) – definují školní rok s klíčovými termíny: datum zadání, odevzdání a zpětné vazby. Projekty a uživatelé jsou přiřazeni k roku.

Předměty (subjects) – obory studia, ke kterým jsou projekty přiřazeny.

Hodnocení (grades) – jednotlivá hodnocení projektů od hodnotitelů. Každé hodnocení odkazuje na hodnotící škálu a obsahuje číselnou hodnotu.

Hodnotící škály (scales) – definice kritérií hodnocení s maximální hodnotou a popisem. Škály jsou seskupeny do sad (scale_sets) s vahou a pořadím zobrazení.

Sady škál (scale_sets) – konfigurovatelné sady hodnotících kritérií pro různé role (vedoucí, oponent). Umožňují flexibilní nastavení hodnocení pro každý akademický rok.

Přílohy (attachments) – soubory nahrané k projektu. Ukládají se metadata (název, cesta v S3, popis), samotné soubory jsou v cloudovém úložišti.

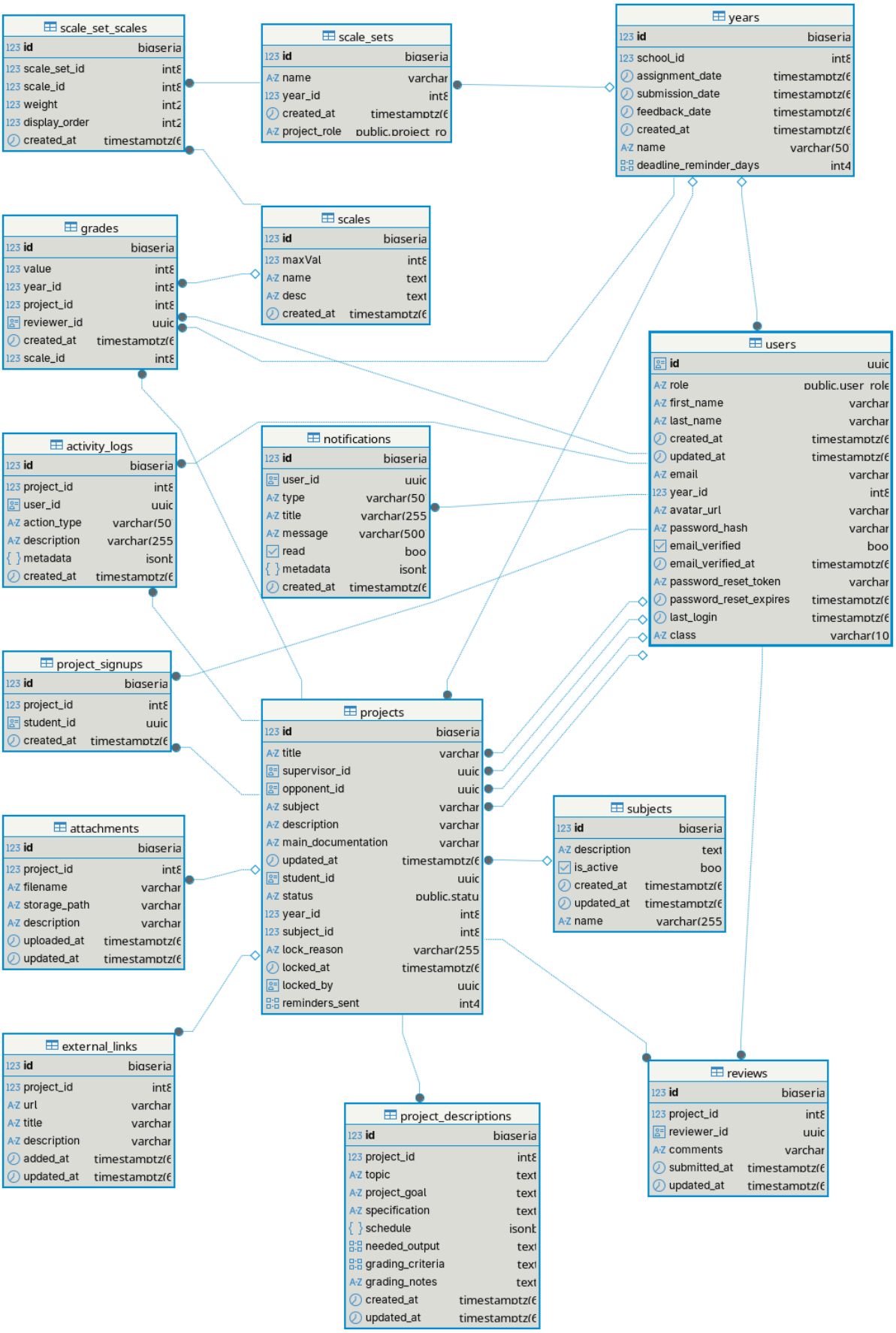
Externí odkazy (external_links) – URL odkazy na externí zdroje projektu (repozitář, dokumentace, demo).

Posudky (reviews) – textové posudky od vedoucího a oponenta k projektům.

Přihlášky studentů (project_signups) – záznam zájmu studentů o volná témata. Každý student se může přihlásit k více projektům, ale vedoucí vybírá jednoho zájemce. Po přiřazení studenta se ostatní přihlášky automaticky odstraní.

Oznámení (notifications) – systémová oznámení pro uživatele. Každé oznámení má typ (přiřazení projektu, odevzdání hodnocení, připomenutí termínu apod.), titulek, zprávu, stav přečtení a volitelná metadata ve formátu JSON.

Protokol aktivit (activity_logs) – záznam všech akcí provedených na projektech pro auditní účely. Vztahy mezi entitami jsou znázorněny na obr. 7.


scale_set_scales
id bigserial
scale_set_id int8
scale_id int8
weight int2
display_order int2
created_at timestamptz(6
scale_sets
id bigserial
name varchar
year_id int8
created_at timestamptz(6
project_role public.project_ro
years
id bigserial
school_id int8
assignment_date timestamptz(6
submission_date timestamptz(6
feedback_date timestamptz(6
created_at timestamptz(6
name varchar(50
deadline_reminder_days int4
grades
id bigserial
value int8
year_id int8
project_id int8
reviewer_id uuid
created_at timestamptz(6
scale_id int8
scales
id bigserial
maxVal int8
name text
desc text
created_at timestamptz(6
users
id uuid
role public.user_role
first_name varchar
last_name varchar
created_at timestamptz(6
updated_at timestamptz(6
email varchar
year_id int8
avatar_url varchar
password_hash varchar
email_verified boo
email_verified_at timestamptz(6
password_reset_token varchar
password_reset_expires timestamptz(6
last_login timestamptz(6
class varchar(10
activity_logs
id bigserial
project_id int8
user_id uuid
action_type varchar(50
description varchar(255
metadata jsonb
created_at timestamptz(6
notifications
id bigserial
user_id uuid
type varchar(50
title varchar(255
message varchar(500
read boo
metadata jsonb
created_at timestamptz(6
project_signups
id bigserial
project_id int8
student_id uuid
created_at timestamptz(6
projects
id bigserial
title varchar
supervisor_id uuid
opponent_id uuid
subject varchar
description varchar
main_documentation varchar
updated_at timestamptz(6
student_id uuid
status public.statu
year_id int8
subject_id int8
lock_reason varchar(255
locked_at timestamptz(6
locked_by uuid
reminders_sent int4
subjects
id bigserial
description text
is_active boo
created_at timestamptz(6
updated_at timestamptz(6
name varchar(255
attachments
id bigserial
project_id int8
filename varchar
storage_path varchar
description varchar
uploaded_at timestamptz(6
updated_at timestamptz(6
external_links
id bigserial
project_id int8
url varchar
title varchar
description varchar
added_at timestamptz(6
updated_at timestamptz(6
project_descriptions
id bigserial
project_id int8
topic text
project_goal text
specification text
schedule jsonb
needed_output text
grading_criteria text
grading_notes text
created_at timestamptz(6
updated_at timestamptz(6
reviews
id bigserial
project_id int8
reviewer_id uuid
comments varchar
submitted_at timestamptz(6
updated_at timestamptz(6

obr. 7: ER diagram datového modelu

3.3 Návrh uživatelského rozhraní

Uživatelské rozhraní systému SubmiTheses je navrženo jako responzivní webová aplikace s důrazem na přehlednost a přizpůsobení obsahu podle role přihlášeného uživatele.

3.3.1 Principy návrhu

Při návrhu rozhraní byly zvoleny následující principy:

- **Responzivní layout** – aplikace se přizpůsobuje šířce obrazovky od mobilních zařízení (320 px) po velké monitory. Využívá tři hlavní breakpointy: 640 px, 768 px a 1024 px.
- **Role-based navigace** – obsah postranního panelu a dostupné akce se dynamicky mění podle role uživatele (admin, učitel, student). Například studenti nevidí sekci správy uživatelů ani administrátorský panel.
- **Podpora tmavého režimu** – uživatel si může zvolit světlý, tmavý nebo systémový režim. Barvy jsou definovány pomocí CSS proměnných s vlastním sémantickým tokenovým systémem (primary, danger, warning, success), což zajišťuje konzistentní vzhled v obou režimech.
- **Dvojjazyčnost** – celé rozhraní je lokalizováno do češtiny a angličtiny pomocí knihovny next-intl. Každá URL adresa obsahuje prefix jazyka (např. `/en/projects`, `/cz/projects`).

3.3.2 Struktura rozložení

Aplikace využívá klasické rozložení dashboardu se dvěma hlavními oblastmi:

- **Postranní panel (sidebar)** – pevná šířka 256 px, obsahuje logo, navigační menu, přepínač motivu, přepínač jazyka a uživatelské menu s odhlášením. Na mobilních zařízeních se panel skrývá a zobrazuje se pomocí hamburger ikony.
- **Hlavní oblast** – zabírá zbývající šířku, obsahuje navigační drobečky (breadcrumbs) a samotný obsah stránky s maximální šířkou 1400 px.

3.3.3 Navigace podle rolí

Postranní panel zobrazuje položky navigace v závislosti na roli uživatele:

Položka	**Admin**	**Učitel**	**Student**
Projekty	✓	✓	✓
Oznámení	✓	✓	✓
Uživatelé	✓	–	–
Nastavení	✓	✓	✓
Admin panel	✓	–	–

tab. 1: Viditelnost navigačních položek podle role

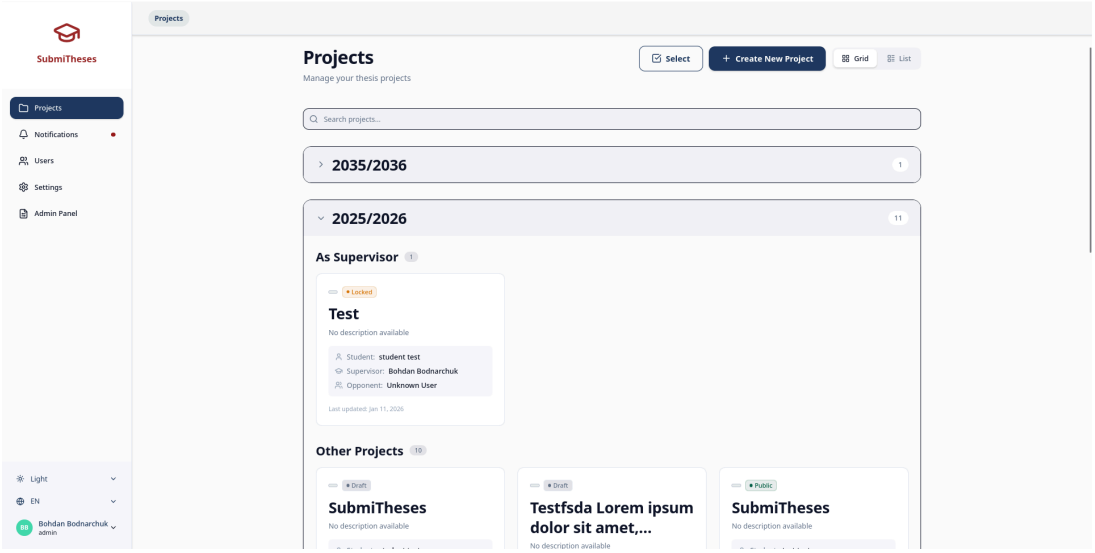
U položky Oznámení se zobrazuje červený indikátor nepřečtených zpráv, který se automaticky aktualizuje každých 60 sekund.

3.3.4 Klíčové obrazovky

3.3.4.1 Seznam projektů

Hlavní stránka zobrazuje projekty rozdělené do sekcí podle vztahu uživatele k projektu: Moje projekty, Jako vedoucí, Jako oponent a Ostatní projekty. Uživatel si může přepínat mezi zobrazením v mřížce a seznamu.

Mřížka využívá responzivní rozložení – 1 sloupec na mobilu, 2 na tabletu a 3 na desktopu.



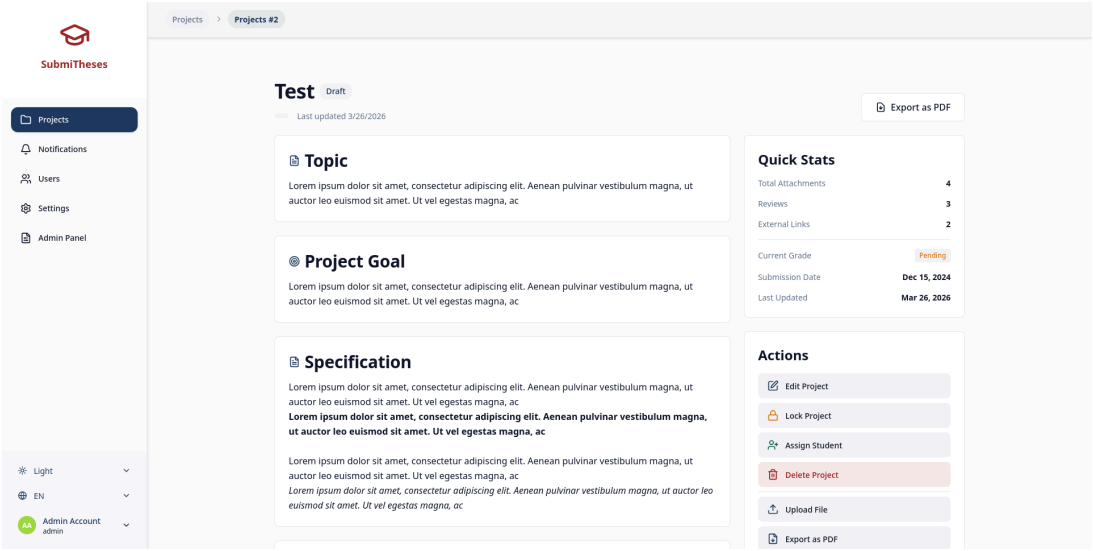
obr. 8: Seznam projektů v mřížkovém zobrazení

3.3.4.2 Detail projektu

Stránka detailu projektu využívá dvousloupcové rozložení na desktopu:

- **Levý sloupec (2/3 šířky)** – přehled projektu (téma, cíl, specifikace, harmonogram, tým) a záložky pro přílohy, externí odkazy a hodnocení.
- **Pravý sloupec (1/3 šířky)** – rychlé statistiky, akční tlačítka (podle role) a nedávná aktivita.

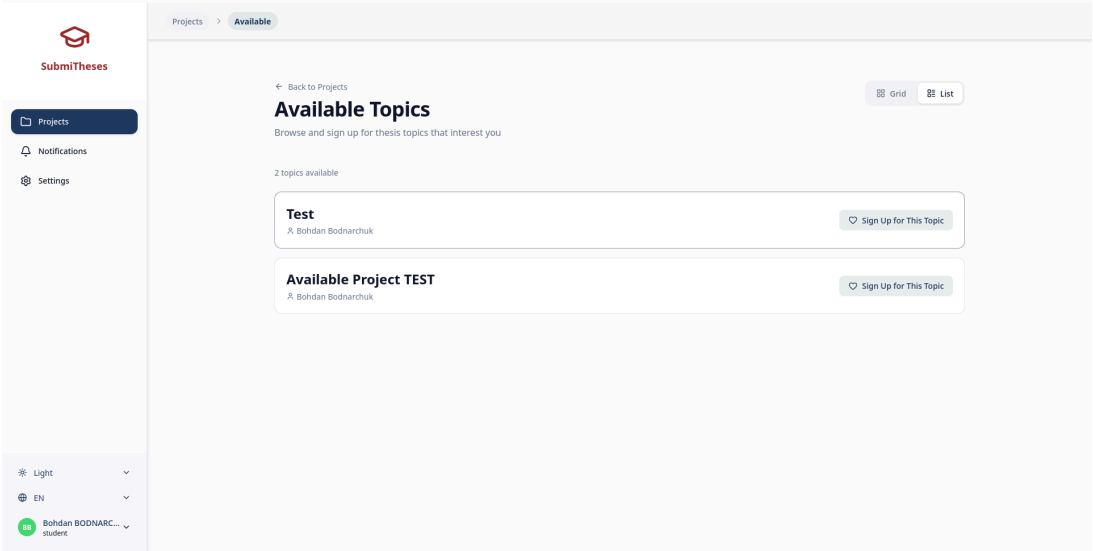
Na mobilních zařízeních se sloupce skládají pod sebe.



obr. 9: Detail projektu – dvousloupcové rozložení

3.3.4.3 Volná témata

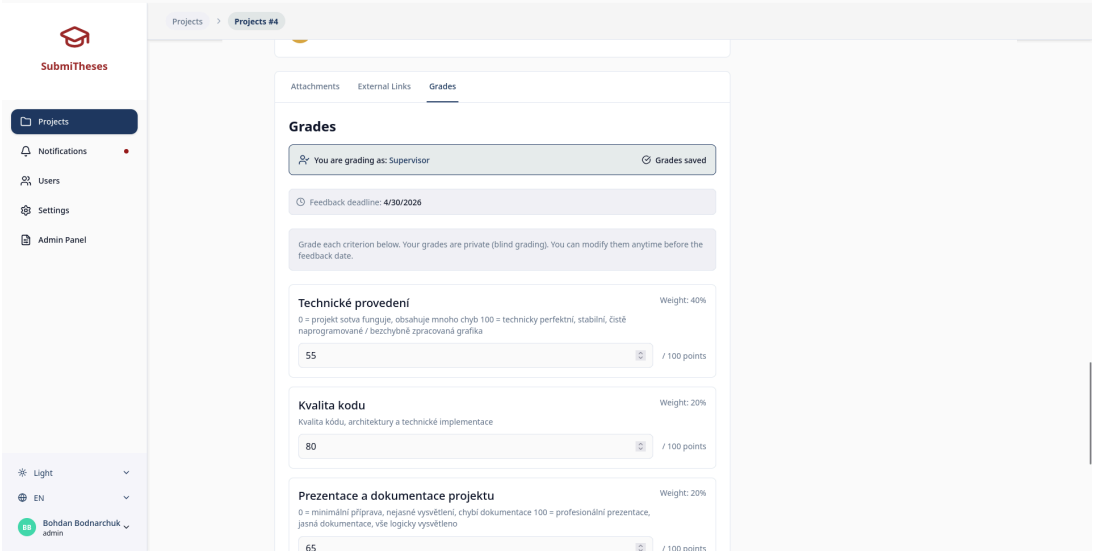
Studenti mají přístup ke stránce s volnými tématy, kde mohou vyjádřit zájem o projekt pomocí tlačítka přihlášení. Vedoucí následně vidí seznam zájemců a může studenta přiřadit.



obr. 10: Stránka volných témat pro studenty

3.3.4.4 Hodnocení

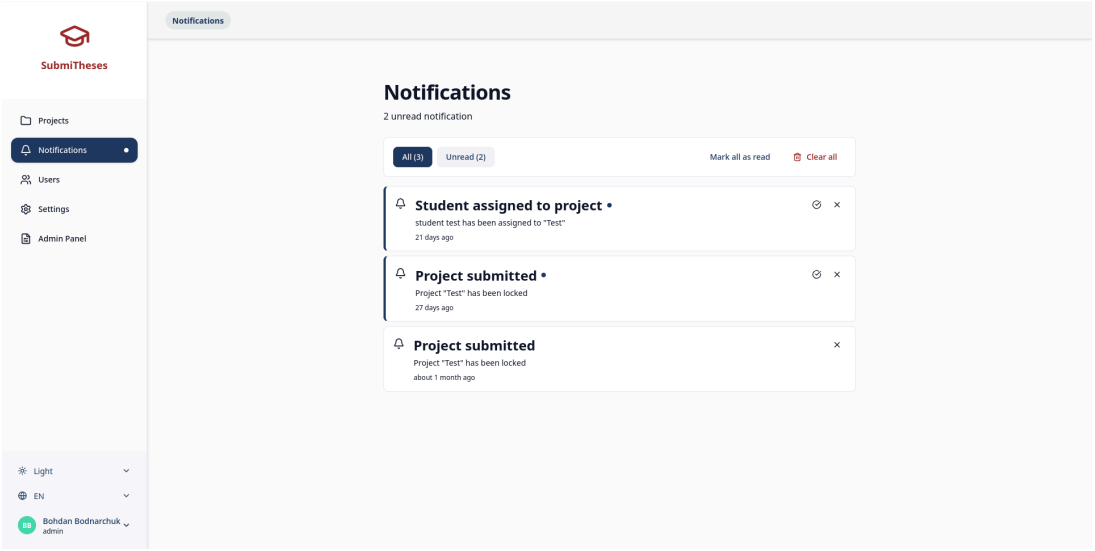
Hodnotící formulář zobrazuje škálu kritérií s maximálními hodnotami a váhami. Učitel zadává hodnoty a systém automaticky vypočítá vážený průměr. Studenti vidí výsledky hodnocení až po datu zpětné vazby stanoveném pro akademický rok.



obr. 11: Formulář pro hodnocení projektu

3.3.4.5 Oznámení

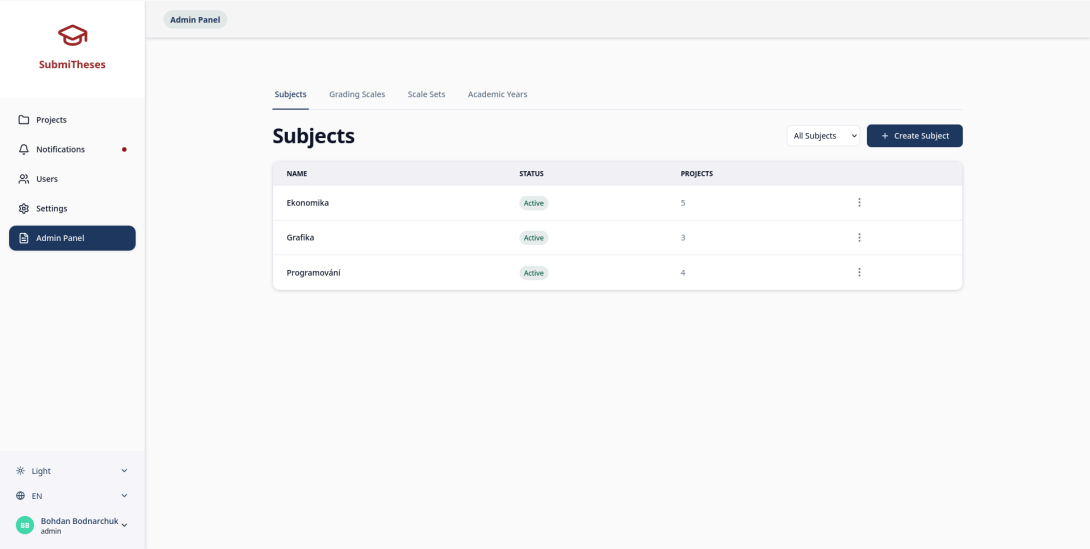
Stránka oznámení zobrazuje seznam notifikací s možností filtrování (všechny / nepřečtené). Nepřečtené zprávy jsou zvýrazněny barevným pruhem na levé straně. Uživatel může označit jednotlivé zprávy nebo všechny najednou jako přečtené.



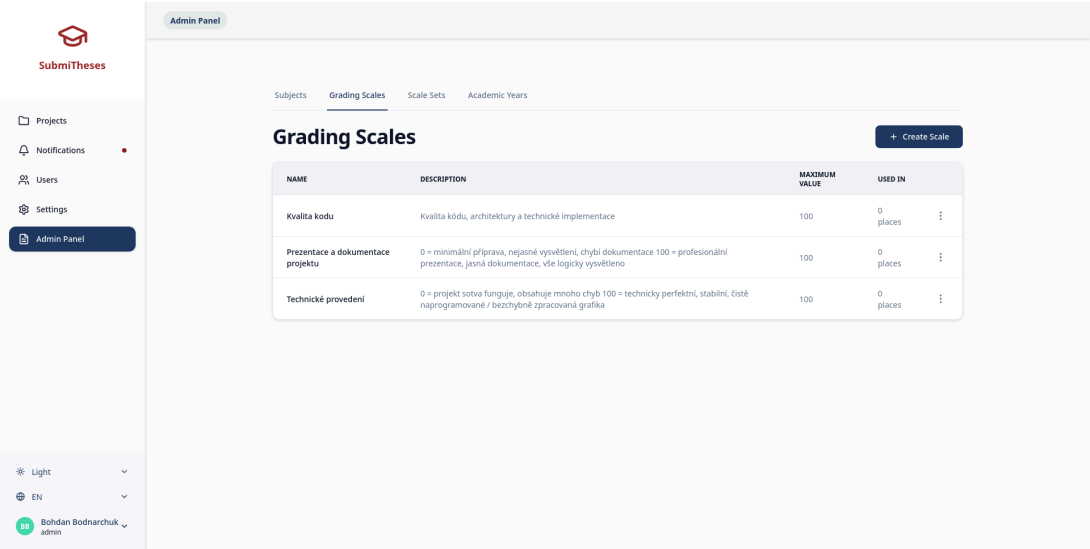
obr. 12: Stránka oznámení s filtrováním

3.3.4.6 Administrátorský panel

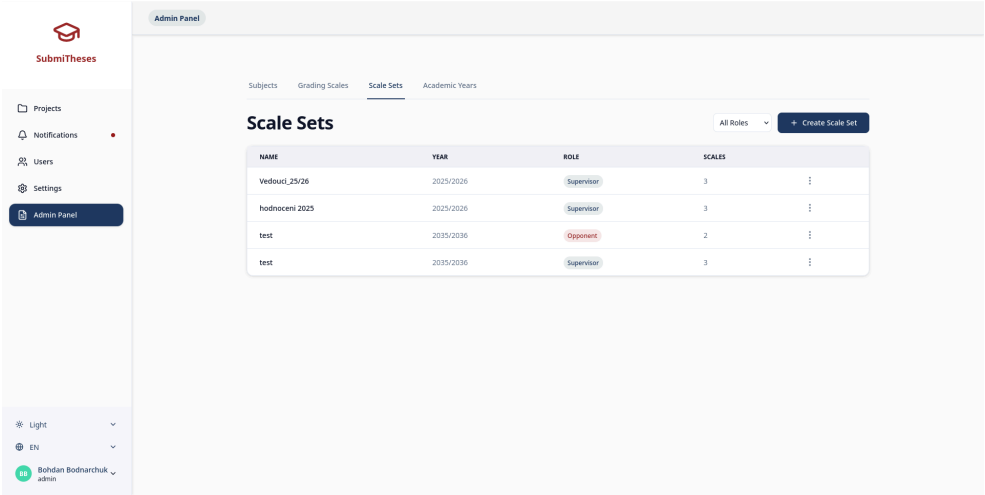
Administrátoři mají přístup k panelu s horizontální záložkovou navigací pro správu předmětů, škál, sad škál a akademických roků. Každá záložka obsahuje datovou tabulku s akcemi pro vytvoření, úpravu a smazání záznamů.



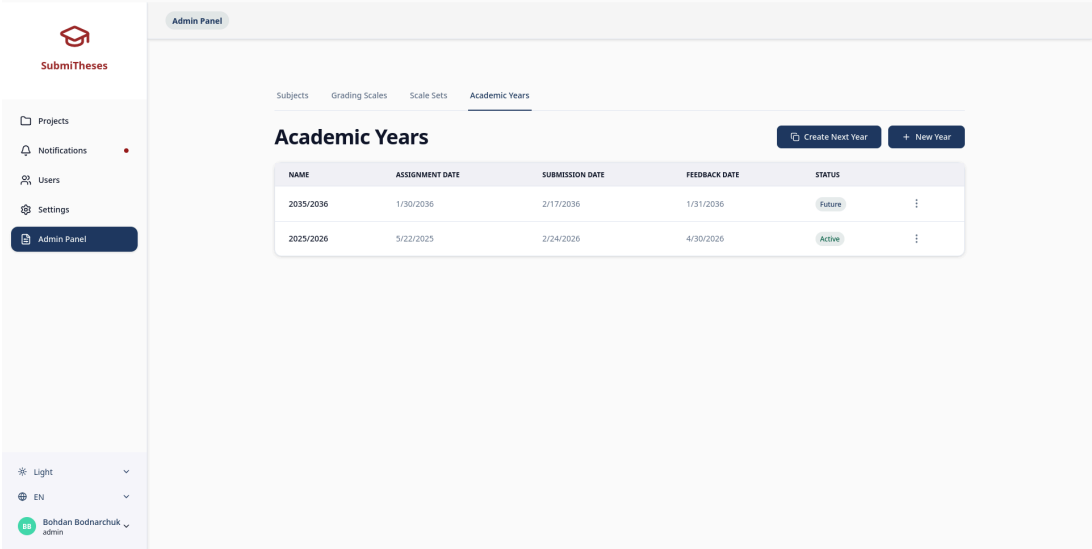
obr. 13: Administrátorský panel — správa předmětů



obr. 14: Administrátorský panel — správa hodnotících škál



obr. 15: Administrátorský panel — správa sad škál



obr. 16: Administrátorský panel — správa akademických roků

3.3.5 Responzivní chování

Aplikace se přizpůsobuje třem hlavním rozlišením:

Rozlišení	Chování
Mobil (< 640 px)	Sidebar skrytý za hamburger ikonou, jednosloupcový layout, zmenšené odsazení
Tablet (768 px+)	Mřížka projektů ve 2 sloupcích, viditelná jména uživatelů
Desktop (1024 px+)	Sidebar vždy viditelný, 3 sloupce mřížky, dvousloupcový detail projektu, plné odsazení

tab. 2: Responzivní chování aplikace

3.4 Implementace

3.4.1 Backendová část

3.4.1.1 Struktura složek

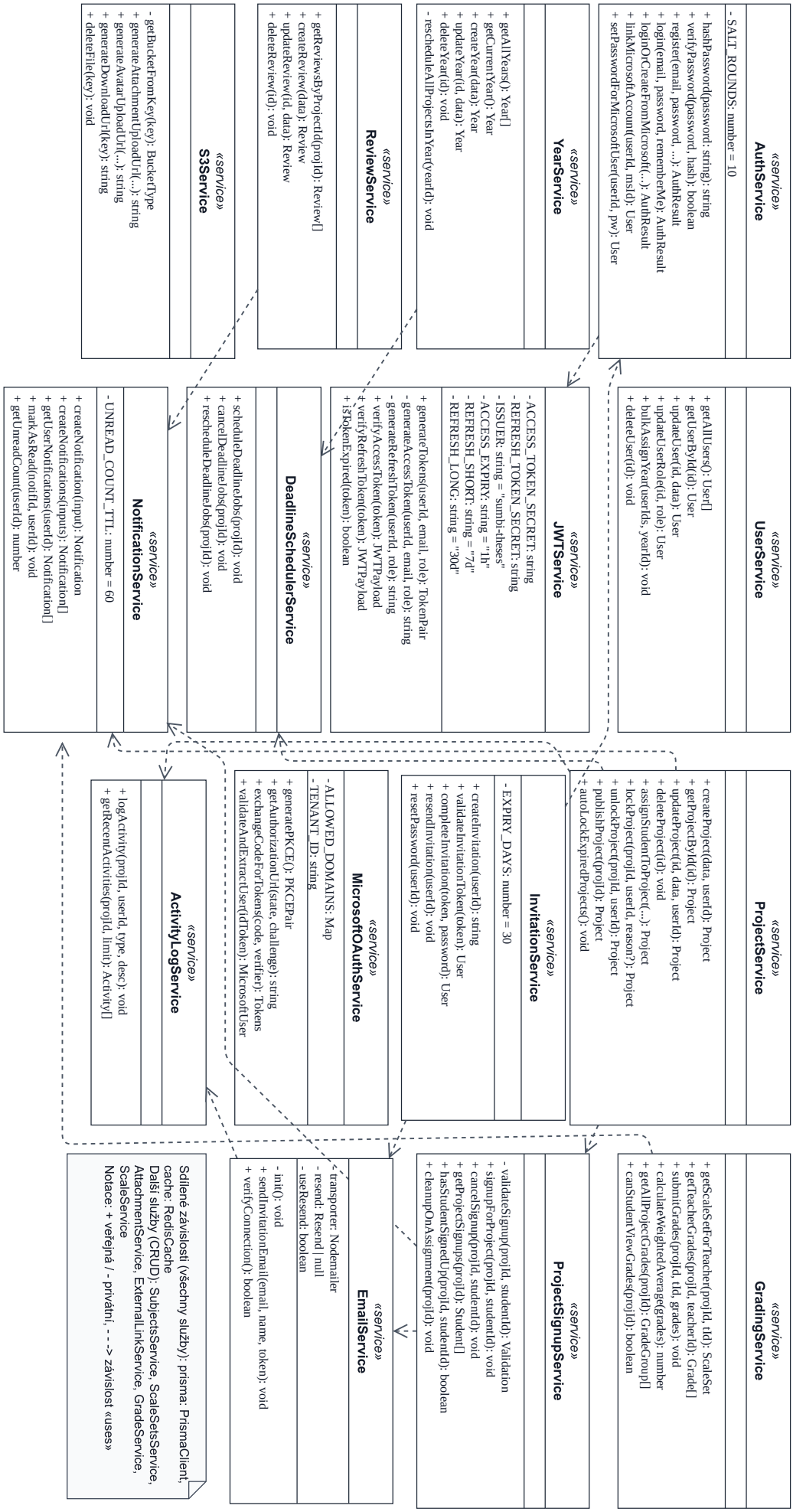
Zdrojový kód backendu je organizován v adresáři `backend/src/` do následujících složek:

- `controllers/` – zpracování HTTP požadavků a volání příslušných služeb. Každý controller odpovídá jedné doménové oblasti (projekty, uživatelé, oznámení apod.).
- `services/` – hlavní aplikační logika oddělená od HTTP vrstvy. Služby obsahují veškerou business logiku a komunikaci s databází přes Prisma ORM.
- `routes/` – definice API tras a napojení na controllery s příslušnými middleware.
- `middleware/` – autentizační a autorizační middleware pro ověření JWT tokenů a kontrolu rolí.
- `validation/` – validační schémata pro ověření vstupních dat pomocí knihovny Zod.

- `queues/` – definice front pro zpracování úloh na pozadí (BullMQ).
- `workers/` – workery zpracovávající úlohy z front (připomenutí termínů, zamykání projektů).
- `config/` – konfigurace aplikace (databáze, S3, Redis, JWT).
- `lib/` – inicializace sdílených knihoven (Prisma klient, Redis připojení).
- `templates/` – šablony pro e-mailové notifikace.
- `types/` – TypeScript typy specifické pro backend.
- `utils/` – pomocné funkce (formátování dat, generování tokenů apod.).

Tato struktura sleduje princip oddělení odpovědností – HTTP vrstva (controllers/routes) je oddělena od business logiky (services), což usnadňuje testování a údržbu kódu. Přehled klíčových servisních tříd a jejich vzájemných závislostí je znázorněn na obr. 17.

Diagram tříd zachycuje hlavní servisní vrstvu backendu, zejména odpovědnosti jednotlivých služeb a vazby mezi nimi. Z diagramu je patrné, které služby tvoří jádro autentizace, správy projektů, notifikací a plánování termínů a jak spolu tyto části spolupracují.



obr. 17: Diagram tříd — servisní vrstva backendu

3.4.1.2 Fronta úloh a plánování termínů

Projekty v systému mají pevně stanovené termíny odevzdání definované v rámci akademického roku. Pro zajištění automatického připomínání blížících se termínů a zamykání projektů po uplynutí deadlinu byl implementován systém zpracování úloh na pozadí pomocí knihovny BullMQ s Redis jako brokerem zpráv.

Systém využívá jednu frontu s názvem `deadlines`, která zpracovává dva typy úloh:

- **reminder** – odešle notifikaci studentovi N dní před termínem odevzdání. Dny připomenutí jsou konfigurovatelné pro každý akademický rok (výchozí hodnoty: 7, 3 a 1 den před termínem). Každé připomenutí je idempotentní – systém si v poli `reminders_sent` u projektu ukládá, která připomenutí již byla odeslána, a duplicitní odeslání přeskočí.
- **lock** – automaticky uzamkne projekt v přesný okamžik deadlinu. Při zamčení se změní stav projektu na `locked`, zaloguje se aktivita a všichni účastníci projektu (student, vedoucí, oponent) obdrží notifikaci.

Plánování úloh probíhá ve třech situacích: při vytvoření projektu (pokud má přiřazeného studenta a akademický rok), při přiřazení studenta k projektu a při změně termínu v akademickém roce. Při změně termínu se existující naplánované úlohy nejprve zruší a poté se naplánují nové odpovídající novému deadlinu.

Pro zajištění spolehlivosti jsou úlohy konfigurovány s 3 opakovanými pokusy při selhání s exponenciálním navyšováním prodlevy (počáteční prodleva 1 minuta). Worker zpracovává až 5 úloh souběžně. Před zpracováním každé úlohy se kontroluje, zda projekt stále existuje, není již zamčen a má přiřazeného studenta.

Pro monitoring fronty je k dispozici administrátorské rozhraní Bull Board dostupné na cestě `/admin/queues`, které zobrazuje stav naplánovaných, probíhajících a dokončených úloh.

3.4.2 Frontendová část

3.4.2.1 Struktura aplikace a routování

Frontend využívá App Router frameworku Next.js s dynamickým segmentem `[locale]` v cestě, který zajišťuje lokalizaci všech URL adres (např. `/en/projects`, `/cz/projects`). Podporovány jsou dva jazyky – čeština a angličtina – s angličtinou jako výchozím jazykem.

Stránky jsou organizovány do dvou skupin tras (route groups):

- **(dashboard)** – chráněné stránky pro přihlášené uživatele, které sdílejí společný layout s postranním panelem a navigací.

- **(auth)** – veřejné stránky pro přihlášení a nastavení hesla z pozvánky, bez postranního panelu.

Lokalizace je implementována pomocí knihovny next-intl. Překlady jsou uloženy v JSON souborech (`locales/en/common.json`, `locales/cz/common.json`) a načítány dynamicky podle aktuálního jazyka. V serverových komponentách se používá funkce `getMessages()`, v klientských komponentách hook `useTranslations()`.

3.4.2.2 Autentizace a řízení přístupu

Ověření identity uživatele probíhá na úrovni Next.js middleware. Middleware ověřuje podpis JWT tokenu pomocí knihovny `jose` přímo na straně frontendu – bez nutnosti volat backend. Tento přístup eliminuje síťovou latenci při kontrole autentizace a umožňuje rychlé rozhodnutí o přístupu ke stránce.

Z JWT payloadu se extrahuje role uživatele a nastaví se do HTTP hlavičky `x-current-role`, kterou následně čte dashboard layout pro vykreslení navigace a obsahu odpovídajícího dané roli. Nepřihlášení uživatelé jsou přesměrováni na přihlašovací stránku, přihlášení uživatelé na stránce `/auth` jsou přesměrováni na seznam projektů.

Řízení přístupu k jednotlivým trasám je definováno v centrálním konfiguračním souboru, kde každá trasa specifikuje povolené role a volitelně kontrolu vlastnictví zdroje (např. student může editovat pouze svůj vlastní profil). Pokud uživatel nemá oprávnění, middleware provede rewrite na komponentu `AccessDenied` bez změny URL adresy.

Pro synchronizaci stavu přihlášení mezi záložkami prohlížeče je využito BroadcastChannel API – odhlášení v jedné záložce se okamžitě projeví ve všech ostatních.

3.4.2.3 Komunikace s backendem

Veškerá komunikace s REST API probíhá přes univerzální funkci `apiRequest<T>()`, která funguje jak v serverových, tak v klientských komponentách. Na straně serveru funkce ručně čte cookies z požadavku a předává je v hlavičce, na straně klienta využívá `credentials: 'include'` pro automatické odesílání cookies.

API volání jsou organizována do modulů podle domény (projekty, uživatelé, oznámení, hodnocení, přílohy apod.), přičemž každý modul exportuje typované funkce. Typy dat jsou sdíleny mezi frontendem a backendem prostřednictvím workspace balíčku `@sumbi/shared-types`, což zajišťuje konzistenci datových struktur na obou stranách.

Pro operace na klientské straně je k dispozici hook `useApi()`, který zapouzdřuje stav načítání, chybový stav a data do jednoduchého rozhraní s manuálním spuštěním přes funkci `execute()`.

3.4.2.4 Oznámení a polling

Systém oznámení na frontendu využívá pravidelné dotazování (polling) s intervalem 60 sekund. Postranní panel zobrazuje červený indikátor u ikony oznámení, pokud existují nepřečtená oznámení. Stránka oznámení podporuje filtrování (všechna / nepřečtená), stránkování a hromadné označení jako přečtené.

4 Dokumentace API

Backend poskytuje REST API pro komunikaci s frontendem. Všechny endpointy kromě přihlášení vyžadují autentizaci, některé operace jsou omezeny na konkrétní role (viz tab. 3).

4.1 Autentizace

Pro ověření identity uživatelů systém využívá JWT (JSON Web Token). Při přihlášení server ověří e-mail a heslo (hashované algoritmem bcrypt) a vygeneruje dva tokeny:

- **Access token** – platnost 1 hodina, slouží k autorizaci požadavků
- **Refresh token** – platnost 7–30 dní, slouží k obnovení access tokenu

Tokeny jsou uloženy v httpOnly cookies, což chrání před XSS útoky – JavaScript v prohlížeči k nim nemá přístup.

Jako alternativu k přihlášení e-mailem a heslem systém podporuje jednotné přihlášení (SSO) prostřednictvím Microsoft OAuth 2.0 (Azure AD / Entra ID). Tok využívá rozšíření PKCE (Proof Key for Code Exchange) pro zabezpečení výměny autorizačního kódu. Po úspěšné autentizaci u Microsoftu systém ověří podpis ID tokenu pomocí JWKS (JSON Web Key Set), zkontroluje doménu e-mailu a automaticky přiřadí roli: doména `delta-studenti.cz` odpovídá roli studenta, `delta-skola.cz` roli učitele. Pokud uživatel s daným e-mailem v systému již existuje, dojde k propojení účtu s Microsoft identitou; v opačném případě je vytvořen nový účet.

4.2 Přehled endpointů

Endpoint	Popis	Oprávnění
Projekty		
GET /projects	Seznam všech projektů	všichni
GET /projects/:id	Detail projektu	všichni
POST /projects	Vytvoření nového projektu	učitel
PUT /projects/:id	Úprava projektu	vlastník
DELETE /projects/:id	Smazání projektu	vlastník
PUT /projects/:id/status	Změna stavu (draft/locked/public)	vlastník

Endpoint	Popis	Oprávnění
PUT /projects/:id/student	Přiřazení studenta k projektu	vedoucí
Přihlašování studentů		
POST /projects/:id/signup	Přihlášení zájmu o téma	student
DELETE /projects/:id/signup	Zrušení přihlášky	student
GET /projects/:id/signups	Seznam zájemců o téma	vedoucí
GET /projects/:id/signup/status	Stav přihlášky aktuálního studenta	student
Hodnocení		
GET /projects/:id/grading/scale-set	Hodnotící škála pro hodnotitele	hodnotitel
GET /projects/:id/grading/my-grades	Moje zadané známky	hodnotitel
POST /projects/:id/grading/submit	Odeslání hodnocení	hodnotitel
GET /projects/:id/grading/all	Všechna hodnocení projektu	všichni
Přílohy		
GET /projects/:id/attachments	Seznam příloh projektu	všichni
POST …/request-upload	Vyžádání URL pro nahrání souboru	vlastník
POST …/confirm-upload	Potvrzení nahrání do S3	vlastník
GET …/:attachmentId/download-url	URL pro stažení přílohy	všichni
DELETE …/:attachmentId	Smazání přílohy	vlastník
Posudky		
GET /projects/:id/reviews	Seznam posudků projektu	učitel
POST /projects/:id/reviews	Vytvoření posudku	učitel
PUT …/reviews/:reviewId	Úprava posudku	učitel
DELETE …/reviews/:reviewId	Smazání posudku	admin
Externí odkazy		
GET /projects/:id/links	Seznam externích odkazů	všichni
POST /projects/:id/links	Přidání odkazu	vlastník
PUT …/links/:linkId	Úprava odkazu	vlastník
DELETE …/links/:linkId	Smazání odkazu	vlastník
Oznámení		
GET /notifications	Seznam oznámení (s filtrováním)	vlastní
GET /notifications/unread-count	Počet nepřečtených oznámení	vlastní
PUT /notifications/mark-all-read	Označení všech jako přečtené	vlastní
PUT /notifications/:id/read	Označení jednoho jako přečtené	vlastní

Endpoint	Popis	Oprávnění
DELETE /notifications/:id	Smazání oznámení	vlastní
Uživatelé		
GET /users	Seznam všech uživatelů	admin
GET /users/:id	Detail uživatele	vlastní
POST /users	Vytvoření uživatele	admin
PUT /users/:id	Úprava profilu	vlastní
DELETE /users/:id	Smazání uživatele	admin
PATCH /users/:id/role	Změna role uživatele	admin
GET /users/by-role?role=	Filtr uživatelů podle role	všichni
POST /users/:id/resend-invitation	Opětovné odeslání pozvánky	admin
Předměty		
GET /subjects	Seznam aktivních předmětů	veřejné
GET /subjects/all/list	Všechny předměty vč. neaktivních	admin
POST /subjects	Vytvoření předmětu	admin
PATCH /subjects/:id	Úprava předmětu	admin
DELETE /subjects/:id	Smazání předmětu	admin
Akademické roky		
GET /years	Seznam akademických roků	všichni
GET /years/current	Aktuální akademický rok	všichni
POST /years	Vytvoření nového roku	admin
PUT /years/:id	Úprava roku	admin
DELETE /years/:id	Smazání roku	admin
Sady hodnotících škál		
GET /scale-sets	Seznam sad škál	všichni
GET /scale-sets/:id	Detail sady škál	všichni
POST /scale-sets	Vytvoření sady	admin
PUT /scale-sets/:id	Úprava sady	admin
DELETE /scale-sets/:id	Smazání sady	admin
POST /scale-sets/:id/scales	Přidání škály do sady	admin
DELETE …/scales/:scaleId	Odebrání škály ze sady	admin
POST /scale-sets/bulk-clone	Hromadné klonování sad do nového roku	admin

Endpoint	Popis	Oprávnění
Autentizace		
POST /auth/login	Přihlášení (e-mail + heslo)	veřejné
POST /auth/logout	Odhlášení	všichni
POST /auth/refresh	Obnovení access tokenu	všichni
GET /auth/session	Informace o aktuální relaci	všichni
GET /auth/user	Profil přihlášeného uživatele	všichni
GET /auth/microsoft	Přihlášení přes Microsoft OAuth	veřejné
GET /auth/microsoft/callback	Callback z Microsoft OAuth	veřejné
GET /users/validate-invitation	Ověření pozvánkového tokenu	veřejné
POST /users/setup-password	Nastavení hesla z pozvánky	veřejné

tab. 3: Přehled API endpointů s popisem

5 Uživatelská příručka

Tato kapitola slouží jako návod pro uživatele systému SubmiTheses. Popisuje postup nasazení aplikace na server a práci s jednotlivými funkcemi systému z pohledu všech tří rolí – studenta, učitele a administrátora.

5.1 Nasazení na server

Systém SubmiTheses je kontejnerizován pomocí Dockeru, což umožňuje konzistentní nasazení na libovolný server s podporou Docker Engine. Nasazení lze provést manuálně nebo automatizovaně prostřednictvím CI/CD pipeline.

5.1.1 Docker konfigurace

Backend aplikace využívá vícefázový (multi-stage) Dockerfile pro optimalizaci velikosti výsledného obrazu:

- **Build fáze** – nainstaluje závislosti, vygeneruje Prisma klienta a zkompiluje TypeScript do JavaScriptu. Jako základní obraz slouží `node:latest`.
- **Produkční fáze** – obsahuje pouze produkční závislosti a zkompilovaný kód. Výsledný obraz je výrazně menší než vývojový.

Celá infrastruktura je definována v souboru `docker-compose.prod.yml`, který orchestruje pět služeb:

Služba	Popis	Port
PostgreSQL 15	Relační databáze pro ukládání dat	5432

Služba	Popis	Port
Redis 7	Cache a message broker pro BullMQ fronty	6379
Backend API	Express.js server s REST API	5000
Garage	S3-kompatibilní úložiště souborů	3900
Frontend	Next.js server (standalone režim)	3000

tab. 4: Služby v Docker Compose konfiguraci

Všechny služby mají nakonfigurované health checky, které zajišťují, že se backend spustí až po úspěšné inicializaci databáze a Redis. Data databáze a Redis jsou perzistentní díky Docker volumes.

Pro produkční nasazení je nutné nastavit proměnné prostředí – zejména `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` (vygenerované pomocí `openssl rand -base64 64`), připojení k databázi, SMTP server a konfiguraci AWS S3 pro ukládání souborů.

5.1.2 Úložiště souborů (Garage)

Pro ukládání příloh projektů a profilových obrázků systém využívá **Garage** – self-hosted S3-kompatibilní objektové úložiště. Konfigurace je definována v souboru `garage.toml` v kořenovém adresáři projektu.

Po prvním spuštění kontejnerů (`docker compose up -d`) je nutné provést jednorázovou inicializaci clusteru:

1. Zjistit ID uzlu:
   ```
   docker exec submitheses-garage /garage status
   ```
2. Přiřadit uzel do rozložení clusteru (nahradit `<node-id>` skutečným ID):
   ```
   docker exec submitheses-garage /garage layout assign -z dc1 -c 1G <node-id>
   ```
3. Aplikovat rozložení:
   ```
   docker exec submitheses-garage /garage layout apply --version 1
   ```
4. Vytvořit API klíč:
   ```
   docker exec submitheses-garage /garage key create sumbitheses-key
   ```
5. Vytvořit buckety a přidělit oprávnění:
   ```
   docker exec submitheses-garage /garage bucket create sumbitheses-
   attachments
   docker exec submitheses-garage /garage bucket create sumbitheses-avatars
   docker exec submitheses-garage /garage bucket allow --read --write --owner
   sumbitheses-attachments --key sumbitheses-key
   ```

```
docker exec submitheses-garage /garage bucket allow --read --write --owner
sumbitheses-avatars --key sumbitheses-key
```

6. Zobrazit přihlašovací údaje a zkopírovat je do `.env`:

   ```
   docker exec submitheses-garage /garage key info sumbitheses-key
   ```

   Hodnotu `Key ID` zapsat do `AWS_ACCESS_KEY_ID` a `Secret key` do `AWS_SECRET_ACCESS_KEY`.

Proměnná	**Popis**	**Výchozí hodnota**
`S3_ENDPOINT`	Interní URL S3 API Garage (Docker síť)	`http://garage:3900`
`S3_PUBLIC_ENDPOINT`	Veřejná URL pro pre-signed URL (přes Nginx)	`https://s3.submitheses.app`
`AWS_REGION`	Region (musí odpovídat `garage.toml`)	`garage`
`AWS_ACCESS_KEY_ID`	ID přístupového klíče Garage	—
`AWS_SECRET_ACCESS_KEY`	Tajný klíč Garage	—
`AWS_S3_BUCKET_ATTACHMENTS`	Bucket pro přílohy projektů	`sumbitheses-attachments`
`AWS_S3_BUCKET_AVATARS`	Bucket pro profilové obrázky	`sumbitheses-avatars`
`USE_S3_STORAGE`	Povolit S3 úložiště	`true`

tab. 5: Proměnné prostředí pro konfiguraci Garage úložiště

Pro ověření funkčnosti stačí nahrát soubor přes aplikaci a následně zkontrolovat obsah bucketu:

```
docker exec submitheses-garage /garage bucket info sumbitheses-attachments
```

V případě problémů s přístupem (chyba `AccessDenied`) je třeba ověřit oprávnění klíče příkazem `garage key info` a zkontrolovat shodu přihlašovacích údajů v `.env`. Pokud pre-signed URL adresy nefungují, je nutné ověřit, že `S3_ENDPOINT` ukazuje na správnou adresu – při běhu v Dockeru `http://garage:3900`, při lokálním vývoji `http://localhost:3900`.

5.1.3 Reverzní proxy (Nginx)

Aplikace vyžaduje Nginx jako reverzní proxy se dvěma server bloky:

- **`submitheses.app`** – frontend (port 3000) a backend API (port 5000)
- **`s3.submitheses.app`** – Garage S3 úložiště (port 3900)

Samostatná subdoména pro S3 je nutná, protože pre-signed URL obsahují podpis zahrnující celou cestu – přepis cesty by podpis zneplatnil. Referenční konfigurace je v adresáři `nginx/`. SSL certifikáty lze získat pomocí Certbot:

```
sudo certbot --nginx -d submitheses.app -d s3.submitheses.app
```

5.1.4 Manuální nasazení

Postup manuálního nasazení na server:

1. Naklonovat repozitář: `git clone <repo-url>`
2. Vytvořit soubor `.env` s produkčními proměnnými prostředí
3. Nastavit Nginx – zkopírovat konfigurační soubory z `nginx/` do `/etc/nginx/sites-enabled/`
4. Vytvořit DNS A záznamy pro `submitheses.app` a `s3.submitheses.app`
5. Získat SSL certifikáty: `sudo certbot --nginx -d submitheses.app -d s3.submitheses.app`
6. Spustit aplikaci: `docker compose -f docker-compose.prod.yml up -d --build`
7. Provést inicializaci Garage (viz sekce Úložiště souborů)
8. Provést databázové migrace: `docker exec submitheses-backend npx prisma migrate deploy`

Pro aktualizaci aplikace stačí stáhnout nové změny (`git pull`), přestavět obrazy a restartovat služby.

5.1.5 CI/CD pomocí GitHub Actions

Pro automatizované nasazení lze využít GitHub Actions workflow, který po každém push do hlavní větve provede:

1. **Build a testy** – zkompiluje kód, spustí validaci typů a testy
2. **Sestavení Docker obrazu** – vytvoří produkční obraz a nahraje ho do container registry (např. GitHub Container Registry)
3. **Nasazení na server** – připojí se k serveru přes SSH a aktualizuje běžící kontejnery

Tento přístup zajišťuje, že na server se dostane pouze kód, který prošel všemi kontrolami, a minimalizuje riziko lidské chyby při manuálním nasazení.

5.2 Použití aplikace

Následující podkapitoly popisují práci s aplikací z pohledu koncového uživatele.

5.2.1 Přihlášení do systému

Uživatel přistupuje k aplikaci prostřednictvím webového prohlížeče na adrese, kde je systém nasazen. Na přihlašovací stránce zadá svůj e-mail a heslo. Účty jsou vytvářeny administrátorem, který uživateli odešle pozvánku e-mailem. Pozvánka obsahuje jednorázový odkaz s tokenem, přes který si uživatel nastaví své heslo při prvním přihlášení. Token má omezenou platnost – pokud vyprší, administrátor může odeslat novou pozvánku. Alternativně se uživatel může přihlásit školním účtem přes Microsoft (tlačítko „Přihlásit se přes Microsoft„). Systém na základě domény e-mailu automaticky určí roli – `@delta-studenti.cz` pro studenty, `@delta-skola.cz` pro učitele.

Po úspěšném přihlášení je uživatel přesměrován na hlavní stránku se seznamem projektů. Přihlášení je udržováno pomocí HTTP cookies – uživatel zůstává přihlášen i po zavření prohlížeče, dokud nevyprší platnost refresh tokenu (7–30 dní).

5.2.2 Přehled projektů

Hlavní stránka aplikace zobrazuje seznam všech projektů, ke kterým má uživatel přístup. Projekty jsou rozděleny do sekcí podle vztahu uživatele k nim:

- **Moje projekty** – projekty, kde je uživatel studentem (zobrazuje se pouze studentům)
- **Jako vedoucí** – projekty, kde je uživatel vedoucím práce (učitelé a admini)
- **Jako oponent** – projekty, kde je uživatel oponentem (učitelé a admini)
- **Ostatní projekty** – všechny ostatní projekty v systému

Uživatel si může přepínat mezi zobrazením v mřížce (karty) a seznamu (tabulka) pomocí ikon v pravém horním rohu. K dispozici je také vyhledávací pole pro filtrování projektů podle názvu. Učitelé a administrátoři mohou z této stránky vytvořit nový projekt pomocí tlačítka „Nový projekt„.

5.2.3 Vytvoření projektu

Vytvoření nového projektu je dostupné pro učitele a administrátory. Proces probíhá formou pětikrokového průvodce:

1. **Základní informace** – název projektu, výběr předmětu a akademického roku
2. **Téma a cíle** – popis tématu práce a stanovení cílů projektu
3. **Specifikace a výstupy** – detailní specifikace zadání a požadované výstupy
4. **Harmonogram** – plánované milníky a termíny průběžných kontrol
5. **Výběr týmu** – přiřazení studenta, vedoucího a oponenta ze seznamu uživatelů

Rozpracovaný projekt se automaticky ukládá jako koncept (draft). Vedoucí může projekt kdykoli upravit, dokud není uzamčen po uplynutí termínu odevzdání.

5.2.4 Volná témata

Studenti mají přístup ke stránce „Volná témata„, která zobrazuje projekty bez přiřazeného studenta. Student si může prohlédnout detail tématu a vyjádřit zájem o projekt kliknutím na tlačítko přihlášení. K jednomu tématu se může přihlásit více studentů.

Vedoucí práce vidí seznam všech přihlášených zájemců a z nich vybere jednoho studenta, kterého přiřadí k projektu. Po přiřazení se ostatní přihlášky automaticky odstraní a přiřazený student obdrží notifikaci.

5.2.5 Detail projektu

Stránka detailu projektu využívá dvousloupcové rozložení:

Levý sloupec (hlavní obsah) zobrazuje přehled projektu s informacemi o tématu, cílech, specifikaci, harmonogramu a přiřazeném týmu. Pod přehledem se nachází záložková navigace pro přístup k:

- **Přílohy** – nahrané soubory projektu
- **Externí odkazy** – odkazy na repozitáře, dokumentaci, demo apod.
- **Posudky** – textové posudky vedoucího a oponenta
- **Hodnocení** – známky podle hodnotících škál

Pravý sloupec obsahuje rychlé statistiky (počet příloh, odkazů, stav hodnocení), akční tlačítka dostupná podle role uživatele a chronologický přehled nedávné aktivity na projektu.

5.2.6 Správa příloh a odkazů

Nahrávání souborů: Na záložce „Přílohy„ v detailu projektu může vlastník (student nebo vedoucí) nahrát soubory kliknutím na tlačítko „Nahrát přílohu“. Systém nejprve vyžádá pre--signed URL od backendu a poté nahraje soubor přímo do S3-kompatibilního úložiště Garage. Povolené typy souborů zahrnují PDF, Word, Excel, obrázky a ZIP archivy s maximální velikostí 10 MB.

Externí odkazy: Na záložce „Externí odkazy„ lze přidávat URL adresy na externí zdroje projektu (např. odkaz na GitHub repozitář, online dokumentaci nebo demo aplikaci). Každý odkaz obsahuje název a URL adresu.

Stahování příloh je dostupné všem přihlášeným uživatelům – systém vygeneruje dočasnou URL adresu pro stažení souboru z S3.

5.2.7 Hodnocení

Hodnocení projektů probíhá prostřednictvím konfigurovatelných hodnotících škál. Vedoucí a oponent mají přiřazeny vlastní sady škál s různými kritérii a váhami.

Učitel (hodnotitel) přistoupí k hodnocení přes záložku „Hodnocení„ v detailu projektu. Formulář zobrazuje seznam kritérií s maximálními hodnotami – učitel zadá číselné hodnocení pro každé kritérium. Systém automaticky vypočítá vážený průměr na základě vah jednotlivých škál v sadě.

Student vidí výsledky hodnocení až po datu zpětné vazby (feedback date), které je nastaveno pro daný akademický rok. Do tohoto data jsou známky skryté.

5.2.8 Oznámení

Stránka oznámení zobrazuje chronologický seznam systémových notifikací. Uživatel obdrží oznámení při událostech jako přiřazení k projektu, odevzdání hodnocení, připomenutí blížícího se termínu nebo uzamčení projektu.

Dostupné akce:

- **Filtrování** – přepínání mezi zobrazením všech a pouze nepřečtených oznámení
- **Označení jako přečtené** – jednotlivě kliknutím na oznámení, nebo hromadně tlačítkem „Označit vše jako přečtené„
- **Smazání** – odstranění jednotlivých oznámení
- **Vymazání všech** – hromadné smazání všech oznámení

V postranním panelu se u ikony oznámení zobrazuje červený indikátor s počtem nepřečtených zpráv, který se automaticky aktualizuje každých 60 sekund.

5.2.9 Nastavení profilu

Na stránce nastavení může uživatel upravit své osobní údaje:

- **Profil** – změna jména a příjmení
- **E-mail** – změna přihlašovacího e-mailu
- **Propojení Microsoft účtu** – uživatelé s lokálním účtem mohou propojit svůj školní Microsoft účet pro jednotné přihlášení (SSO)
- **Heslo** – změna hesla (vyžaduje zadání současného hesla a dvakrát nového), případně nastavení nového hesla pro uživatele přihlášené přes Microsoft

5.2.10 Administrátorský panel

Administrátorský panel je přístupný pouze uživatelům s rolí admin. Panel obsahuje čtyři záložky pro správu systémových dat:

- **Předměty** – správa studijních oborů (název, aktivní/neaktivní stav). Neaktivní předměty nejsou nabízeny při vytváření nových projektů.
- **Škály** – správa jednotlivých hodnotících kritérií s maximální hodnotou a popisem.
- **Sady škál** – konfigurace sad hodnotících kritérií s vahou a pořadím zobrazení. Sady lze přiřadit k akademickému roku a roli hodnotitele (vedoucí/oponent). K dispozici je funkce hromadného klonování sad do nového akademického roku.
- **Akademické roky** – správa školních roků s klíčovými termíny (datum zadání, odevzdání, zpětné vazby) a konfigurací dnů připomenutí.

Administrátor má dále přístup ke správě uživatelů, kde může vytvářet nové účty, měnit role uživatelů, odesílat pozvánky a mazat účty. K dispozici je také funkce hromadné publikace projektů.

5.2.11 Veřejná galerie

Publikované projekty jsou přístupné veřejně bez nutnosti přihlášení. Veřejná galerie zobrazuje projekty se stavem „public„, seskupené podle akademického roku. Návštěvníci mohou procházet projekty, filtrovat je pomocí vyhledávání a přepínat mezi zobrazením v mřížce a seznamu. U veřejných projektů jsou viditelné základní informace (název, téma, tým), přílohy ke stažení a hodnocení. Toto umožňuje prezentaci nejlepších prací a slouží jako inspirace pro budoucí studenty.

6 Bezpečnost

Bezpečnost webové aplikace je klíčovým aspektem návrhu systému SubmiTheses. Aplikace implementuje vícevrstvou ochranu proti nejčastějším typům útoků definovaným v OWASP Top 10.

6.1 SQL Injection

SQL injection je technika, při které útočník vkládá škodlivý SQL kód do vstupních polí aplikace s cílem manipulovat databázové dotazy. Systém SubmiTheses je proti tomuto útoku chráněn použitím **Prisma ORM**, který automaticky parametrizuje všechny databázové dotazy. To znamená, že uživatelský vstup nikdy není přímo vkládán do SQL řetězce – místo toho je předán jako parametr, čímž je znemožněno jeho interpretování jako SQL příkazu.

6.2 Cross-Site Scripting (XSS)

XSS útoky spočívají ve vložení škodlivého JavaScriptu do webové stránky, který se poté spustí v prohlížeči ostatních uživatelů. Ochrana je zajištěna na několika úrovních:

- **Helmet.js** – middleware, který nastavuje bezpečnostní HTTP hlavičky včetně Content Security Policy (CSP), X-Content-Type-Options, X-Frame-Options a dalších. Tyto hlavičky instruují prohlížeč, aby omezil spouštění neautorizovaných skriptů.
- **httpOnly cookies** – JWT tokeny jsou uloženy v cookies s příznakem `httpOnly`, který zabraňuje přístupu k nim z JavaScriptu. I v případě úspěšného XSS útoku tak útočník nemůže získat autentizační tokeny.
- **SameSite cookies** – atribut `sameSite: 'lax'` omezuje odesílání cookies při požadavcích z jiných domén.

6.3 Cross-Site Request Forgery (CSRF)

CSRF útok nutí přihlášeného uživatele nechtěně provést akci na webové aplikaci. Systém implementuje ochranu pomocí vlastního CSRF middleware, který u všech stavově měnících požadavků (POST, PUT, DELETE, PATCH) na API endpointy vyžaduje přítomnost hlavičky `X-Requested-With: XMLHttpRequest`. Prohlížeče neumožňují cizím webům nastavovat vlastní hlavičky při cross-origin požadavcích, čímž je útok znemožněn.

6.4 Zabezpečení OAuth

Přihlášení přes Microsoft OAuth 2.0 implementuje několik bezpečnostních mechanismů:

- **PKCE (Proof Key for Code Exchange)** – při zahájení přihlášení server vygeneruje náhodný `code_verifier` a jeho SHA-256 hash (`code_challenge`). Autorizační server Microsoft přijme kód pouze při předložení původního verifieru, čímž se zabraňuje zachycení autorizačního kódu třetí stranou.
- **Parametr state** – náhodná hodnota zaslaná v autorizačním požadavku a ověřená v callbacku chrání proti CSRF útokům na OAuth tok.
- **Ověření podpisu ID tokenu** – ID token vrácený Microsoftem je ověřen pomocí veřejných klíčů z JWKS (JSON Web Key Set) endpointu Microsoftu. Tím je zaručeno, že token nebyl podvržen.
- **Validace domény** – systém přijímá pouze e-maily z povolených domén (`delta-studenti.cz`, `delta-skola.cz`), čímž zabraňuje přihlášení neautorizovaným Microsoft účtům.

6.5 Ochrana hesel

Uživatelská hesla jsou hashována algoritmem **bcrypt** s 10 koly solení. Bcrypt je záměrně pomalý algoritmus navržený pro hashování hesel – i při úniku databáze je zpětné získání hesel výpočetně nereálné. Systém nikdy neukládá hesla v čitelné podobě.

6.6 Rate Limiting

Pro ochranu proti útokům hrubou silou (brute-force) jsou citlivé endpointy chráněny rate limiterem (knihovna `express-rate-limit`):

Endpoint	Limit	Časové okno
Přihlášení	10 pokusů	15 minut
Registrace	5 pokusů	1 hodina
Obnovení tokenu	30 pokusů	15 minut
OAuth přihlášení	30 pokusů	15 minut

tab. 6: Konfigurace rate limitingu pro autentizační endpointy

Po dosažení limitu server vrací chybový kód 429 (Too Many Requests) a uživatel musí vyčkat do uplynutí časového okna.

6.7 Validace vstupních dat

Veškerá vstupní data z HTTP požadavků jsou validována pomocí knihovny **Zod** ještě před zpracováním v aplikační logice. Validační schémata definují přesný formát očekávaných dat – typy, délky řetězců, formáty e-mailů, rozsahy čísel a povolené hodnoty výčtových typů. Nevalidní požadavky jsou okamžitě odmítnuty s chybovým kódem 400 a popisem chyby.

6.8 CORS (Cross-Origin Resource Sharing)

CORS politika omezuje, z jakých domén může frontend komunikovat s backendem. Povolené origin adresy jsou konfigurovány prostřednictvím proměnné prostředí `CORS_ORIGIN`. V produkci je povolen přístup pouze z domény frontendu, čímž se zabraňuje neoprávněným požadavkům z jiných webů.

6.9 Bezpečnost souborů

Nahrávání souborů je zabezpečeno na několika úrovních:

- **Whitelist MIME typů** – povoleny jsou pouze definované typy souborů (PDF, Word, Excel, obrázky, ZIP). Ostatní typy jsou odmítnuty.
- **Omezení velikosti** – přílohy max. 10 MB, profilové obrázky max. 2 MB.

- **Sanitizace názvů** – názvy souborů jsou zpracovány funkcí `path.basename()`, která odstraní cestu a zabrání útokům typu directory traversal.
- **Pre-signed URL** – soubory se nahrávají přímo do Garage přes dočasné URL s platností 5 minut, čímž se zamezuje přímému průchodu souborů přes server.

6.10 Autorizace a řízení přístupu

Systém implementuje role-based access control (RBAC) prostřednictvím sady autorizačních middleware. Každý API endpoint má přiřazeny povolené role (admin, teacher, student) a volitelně kontrolu vlastnictví zdroje – například student může upravovat pouze svůj vlastní profil a projekt. Middleware navíc detekuje zastaralé JWT tokeny porovnáním role v tokenu s aktuální rolí v databázi.

6.11 Statická analýza kódu (CodeQL)

Pro automatickou detekci bezpečnostních zranitelností využívá projekt nástroj CodeQL, integrovaný přímo v platformě GitHub. CodeQL provádí sémantickou analýzu zdrojového kódu a dokáže odhalit vzory vedoucí k SQL injection, XSS, nezabezpečeným závislostem či chybám v autentizační logice. Na rozdíl od běžných linterů analyzuje tok dat napříč funkcemi, čímž zachytí i nepřímé zranitelnosti.

Skenování je aktivováno v nastavení repozitáře (Code security → Code scanning) a spouští se automaticky při každém push a pull requestu. Pokrývá jak backendový, tak frontendový TypeScript kód, a poskytuje tak průběžnou kontrolu kvality a bezpečnosti napříč celou aplikací.

7 Výsledky

Výsledkem práce je funkční webová aplikace SubmiTheses pro správu a odevzdávání maturitních projektů. Systém pokrývá celý životní cyklus projektu – od vytvoření tématu a přihlášení studentů, přes správu příloh a externích odkazů, až po hodnocení a publikaci hotových prací.

Backend aplikace poskytuje přibližně 60 REST API endpointů organizovaných do 10 doménových oblastí (projekty, uživatelé, hodnocení, přílohy, oznámení, autentizace, předměty, akademické roky, sady škál a přihlášky studentů). Systém rozlišuje tři uživatelské role – administrátor, učitel a student – s granulárním řízením přístupu na úrovni jednotlivých endpointů.

Mezi hlavní implementované funkce patří:

- **Správa projektů** – vytváření projektů pomocí pětikrokového průvodce, úpravy, přiřazování studentů ze seznamu zájemců a změny stavu (draft, locked, public).

- **Hodnotící systém** – konfigurovatelné sady hodnotících škál s váhami, automatický výpočet váženého průměru a oddělené sady pro vedoucího a oponenta.
- **Plánovač termínů** – automatické připomínky blížících se deadlinů (konfigurovatelné dny) a zamykání projektů po uplynutí termínu odevzdání, realizované pomocí BullMQ fronty úloh.
- **Systém oznámení** – notifikace při přiřazení k projektu, odevzdání hodnocení, připomenutí termínu a dalších událostech.
- **Veřejná galerie** – publikované projekty přístupné bez přihlášení, seskupené podle akademického roku.
- **Cloudové úložiště** – nahrávání příloh přes pre-signed URL přímo do Garage bez zatížení serveru.
- **Dvojjazyčné rozhraní** – kompletní lokalizace do češtiny a angličtiny.
- **Tmavý režim** – podpora světlého, tmavého a systémového barevného motivu.

Aplikace je kontejnerizována pomocí Dockeru a připravena k nasazení na libovolný server s podporou Docker Engine.

8 Diskuse

8.1 Zdůvodnění technologických voleb

Volba **Next.js** jako frontendového frameworku byla motivována podporou server-side renderingu a souborového systému pro routování, což zjednodušuje strukturu projektu oproti klasickému SPA přístupu (např. Vite + React Router). Alternativou by byl framework NuxtJS (Vue.js), který nabízí podobné funkce, avšak React ekosystém disponuje rozsáhlejší komunitou a větším množstvím dostupných knihoven.

Express.js byl zvolen pro svou minimalistickou architekturu a flexibilitu. Oproti frameworku NestJS, který nabízí silnější strukturu a dependency injection, je Express jednodušší na nastavení a vhodnější pro projekty střední velikosti. Pro tento systém není režie NestJS opodstatněná.

PostgreSQL jako relační databáze byla preferována před MongoDB, protože datový model systému obsahuje silné vazby mezi entitami (uživatelé, projekty, hodnocení, škály). Relační databáze s cizími klíči a transakcemi zajišťuje konzistenci dat lépe než dokumentová databáze.

8.2 Co se osvědčilo

Monorepo struktura s workspace balíčkem `@sumbi/shared-types` umožnila sdílení TypeScript typů mezi frontendem a backendem, čímž se eliminovaly chyby způsobené nekonzistentními datovými strukturami.

Prisma ORM výrazně zrychlil vývoj díky automatickým migracím, typově bezpečným dotazům a přehledné definici schématu. Nevýhodou je mírně vyšší režie při startu aplikace kvůli generování klienta.

BullMQ s Redis se ukázal jako spolehlivé řešení pro plánování termínů. Konfigurovatelné dny připomenutí a idempotentní zpracování úloh zajišťují správné fungování i při restartech serveru.

Tailwind CSS s Flowbite umožnil rychlé prototypování responzivního rozhraní s podporou tmavého režimu bez nutnosti psát vlastní CSS.

8.3 Omezení a prostor pro zlepšení

Stránka nastavení profilu zatím implementuje pouze základní funkce (změna jména, e-mailu a hesla). Plánované rozšíření o nahrávání profilového obrázku a pokročilejší správu účtu nebylo v rámci projektu dokončeno.

Autentizace podporuje jak e-mail s heslem (pozvánkový systém), tak jednotné přihlášení přes Microsoft OAuth 2.0 (Office 365 SSO). Integrace s Azure AD umožňuje přihlášení školním účtem s automatickým přiřazením role na základě e-mailové domény.

Systém oznámení využívá polling s intervalem 60 sekund. Pro okamžité doručování notifikací by bylo vhodnější implementovat WebSocket spojení, které by eliminovalo zpoždění a snížilo počet HTTP požadavků.

8.4 Srovnání s existujícími řešeními

Oproti obecným LMS systémům (Moodle, Google Classroom) nabízí SubmiTheses specializované funkce přímo navržené pro životní cyklus maturitních prací – konfigurovatelné hodnotící škály s váhami, automatické připomínky termínů, přihlašování studentů k volným tématům a veřejnou galerii hotových prací. Tyto funkce by v obecných systémech vyžadovaly rozsáhlé přizpůsobení pomocí pluginů nebo vlastního vývoje.

9 Závěr

Cílem této práce bylo navrhnout a implementovat webový informační systém pro správu a odevzdávání maturitních projektů. Tento cíl byl splněn – výsledkem je funkční aplikace

SubmiTheses, která pokrývá celý životní cyklus maturitní práce od vytvoření tématu až po publikaci hotového projektu.

Systém implementuje všechny klíčové požadavky ze zadání: správu uživatelů s třemi rolemi, vytváření a správu projektů, nahrávání příloh do cloudového úložiště, hodnocení pomocí konfigurovatelných škál, systém oznámení a automatické připomínky termínů. Aplikace je nasaditelná pomocí Docker kontejnerů a připravena k produkčnímu použití.

Mezi možná rozšíření systému do budoucna patří:

- **Mobilní aplikace** – nativní aplikace pro iOS a Android využívající existující REST API pro pohodlnější přístup z mobilních zařízení.
- **AI asistované hodnocení** – integrace jazykového modelu pro návrhy hodnocení na základě obsahu odevzdané práce, které by hodnotiteli usnadnily orientaci v rozsáhlejších projektech.

Literatura

1. Next.js Documentation. Vercel, 2024. Dostupné z: https://nextjs.org/docs
2. Express.js – Fast, unopinionated, minimalist web framework for Node.js. OpenJS Foundation, 2024. Dostupné z: https://expressjs.com
3. PostgreSQL: The World's Most Advanced Open Source Relational Database. The PostgreSQL Global Development Group, 2024. Dostupné z: https://www.postgresql.org/docs
4. Prisma ORM Documentation. Prisma Data, Inc., 2024. Dostupné z: https://www.prisma.io/docs
5. Redis Documentation. Redis Ltd., 2024. Dostupné z: https://redis.io/docs
6. BullMQ – Premium Message Queue for Node.js based on Redis. Taskforce.sh Inc., 2024. Dostupné z: https://docs.bullmq.io
7. Jones, M.; Bradley, J.; Sakimura, N. RFC 7519 – JSON Web Token (JWT). Internet Engineering Task Force, 2015. Dostupné z: https://datatracker.ietf.org/doc/html/rfc7519
8. OWASP Top Ten – The Ten Most Critical Web Application Security Risks. OWASP Foundation, 2021. Dostupné z: https://owasp.org/www-project-top-ten
9. Tailwind CSS Documentation. Tailwind Labs, Inc., 2024. Dostupné z: https://tailwindcss.com/docs
10. Flowbite – UI Component Library for Tailwind CSS. Bergside Inc., 2024. Dostupné z: https://flowbite.com/docs
11. Zod – TypeScript-first schema validation. Colin McDonnell, 2024. Dostupné z: https://zod.dev

12. Nodemailer – Send emails from Node.js. Andris Reinman, 2024. Dostupné z: https://nodemailer.com
13. Garage – An open-source distributed object storage service. Deuxfleurs, 2024. Dostupné z: https://garagehq.deuxfleurs.fr/documentation
14. Docker Documentation. Docker Inc., 2024. Dostupné z: https://docs.docker.com

Seznam obrázků

obr. 1 Schéma architektury 9
obr. 2 Případy užití — Veřejný přístup a autentizace (UC01–UC08) 10
obr. 3 Případy užití — Správa projektů (UC09–UC17) 11
obr. 4 Případy užití — Hodnocení a klasifikace (UC18–UC24) 12
obr. 5 Případy užití — Administrace (UC25–UC32) 13
obr. 6 Případy užití — Společné a automatizované procesy (UC33–UC36) 14
obr. 7 ER diagram datového modelu 16
obr. 8 Seznam projektů v mřížkovém zobrazení 18
obr. 9 Detail projektu – dvousloupcové rozložení 19
obr. 10 Stránka volných témat pro studenty 19
obr. 11 Formulář pro hodnocení projektu 20
obr. 12 Stránka oznámení s filtrováním 20
obr. 13 Administrátorský panel — správa předmětů 21
obr. 14 Administrátorský panel — správa hodnotících škál 21
obr. 15 Administrátorský panel — správa sad škál 21
obr. 16 Administrátorský panel — správa akademických roků 22
obr. 17 Diagram tříd — servisní vrstva backendu 24

Seznam tabulek

tab. 1 Viditelnost navigačních položek podle role 17
tab. 2 Responzivní chování aplikace 22
tab. 3 Přehled API endpointů s popisem 27
tab. 4 Služby v Docker Compose konfiguraci 30
tab. 5 Proměnné prostředí pro konfiguraci Garage úložiště 32
tab. 6 Konfigurace rate limitingu pro autentizační endpointy 39

Přílohy

9.1 Příloha A: Zdrojový kód aplikace

Kompletní zdrojový kód aplikace SubmiTheses je dostupný v Git repozitáři:

https://github.com/krxgq/SubmiTheses

Repozitář obsahuje monorepo strukturu s adresáři `frontend/` (Next.js aplikace), `backend/` (Express.js API server), `shared-types/` (sdílené TypeScript typy) a `docs/` (dokumentace).

9.2 Příloha B: Pokyny k nasazení

Pro nasazení aplikace je nutné:

1. Nainstalovat Docker Engine a Docker Compose na cílový server
2. Naklonovat repozitář a vytvořit soubor `.env` s konfigurací (databáze, JWT secrets, SMTP, Garage S3)
3. Nastavit Nginx reverzní proxy – zkopírovat konfigurace z `nginx/` do `/etc/nginx/sites-enabled/`
4. Vytvořit DNS A záznamy pro doménu a S3 subdoménu
5. Získat SSL certifikáty pomocí Certbot
6. Spustit příkazem: `docker compose -f docker-compose.prod.yml up -d --build`
7. Provést migrace databáze: `docker exec submitheses-backend npx prisma migrate deploy`
8. Inicializovat Garage úložiště: přiřadit uzel, vytvořit buckety a API klíč (viz kapitola 5.1.2)

Podrobný postup nasazení je popsán v kapitole 5.1 této dokumentace.